AI Driven Development

AI 주도 개발

AI 주도 개발 개요

학습목표

- AI Driven Development의 개념을 이해하고, Gemini CLI 개발 환경을 구축한다

Course Overview

- AI Driven Development vs 전통 개발 방법론 비교
- Vibe Coding → AI Assisted → AI Driven 발전 단계
- Gemini CLI 소개
- 실습

AI Driven Development vs 전통 개발 방법론

구분	전통 개발	AI Driven 개발
개발 방식	• 개발자가 직접 코드 작성 • 수동 설계·구현·테스트	• AI가 코드 생성, 개발자가 검증 • 자연어 프롬프트 기반 개발
개발 속도	• 요구사항 → 설계 → 구현 → 테스트 • 수주~수개월 단위 사이클	• 프롬프트 → 생성 → 검증 → 배포 • 수시간~수일 단위 사이클
개발자 역할	• 코드 작성자 (Coder) • 모든 구현을 직접 수행	• 아키텍트 & 감독자 (Supervisor) • AI 출력물 검증·품질 관리
품질 관리	• 코드 리뷰, 수동 테스트 • 경험에 의존한 버그 탐지	• AI 자동 테스트 생성 + 인간 검증 • 패턴 기반 취약점 자동 탐지
핵심 역량	• 프로그래밍 언어 숙련도 • 알고리즘·자료구조 전문성	• 프롬프트 엔지니어링 능력 • 아키텍처 설계·AI 출력 검증
적합 분야	• 높은 보안 요구 시스템 • 맞춤형 솔루션 개발	• 빠른 프로토타이핑·MVP 개발 • 반복적 업무 자동화

핵심 인사이트: 개발자 역할의 변화

패러다임 전환: 코더에서 아키텍트로

- AI 시대의 개발자는 코드 작성자가 아닌 **AI 시스템의 감독자**로 진화
- 핵심 역량: 프롬프트 엔지니어링, 아키텍처 설계, AI 출력 검증

AI Driven 개발의 장점

- 반복 작업 자동화로 개발자는 창의적 문제 해결에 집중
- 개발 사이클 단축: 수주~수개월 → 수시간~수일
- AI 자동 테스트 생성으로 불량률 감소

주의사항

- AI 생성 코드의 보안 취약점은 인간 코드 대비 높음
- AI를 도구로 활용하되, 검증과 품질 관리는 반드시 인간이 수행

AI 개발의 세 가지 모드

Mode 1: AI Assisted

정의
AI가 코드를 제안하고,
개발자가 검토·수정 후 수락

인간 개입: 높음 (개발자 주도)

특징
- 코드 완성·보일러플레이트 생성
- 개발자 주도 + AI 보조
- 구조화된 프레임워크 내 AI 활용

적합: 팀 협업, 상용 서비스
도구: GitHub Copilot, Cursor, etc

Mode 2: AI Driven

정의
AI가 전체 SDLC 주도,
인간은 검증·의사결정

인간 개입: 중간 (감독 위주)

특징
- AI가 계획·구현·테스트 수행
- 인간은 검증·의사결정에 집중
- 전체 SDLC에 AI 통합

적합: 엔터프라이즈 규모 개발
도구: Claude Code, Antigravity,
Cursor Agent, Gemini CLI, etc

Mode 3: Vibe Coding

정의
자연어로 AI에게 지시하고
코드 검토 최소화

인간 개입: 최소 (결과 확인 위주)

특징
- 빠른 프로토타이핑 / PoC
- 비개발자도 사용 가능
- 보안·품질 리스크 높음

적합: MVP, 개인 프로젝트
도구: Replit Agent, Lovable,
Claude Artifacts, etc

Gemini CLI 소개: 무료 AI 코딩 도구

Gemini CLI란?

- Google이 개발한 **오픈소스 AI 코딩 에이전트**
- 터미널에서 자연어로 코드 생성, 디버그, 파일 조작, 워크플로우 자동화

무료 사용 조건 및 장점

- 모델 제한 + **100만 토큰 컨텍스트 윈도우**
- 분당 60회, 일일 1,000회 요청 무료 (Google 계정만 필요)
- Google Search 그라운딩, MCP 서버 연동, VS Code 통합 지원

Context window

- Gemini는 토큰 100만 개를 수용 가능합니다.
- 실제로 토큰 100만 개는 다음과 같이 표시됩니다.
  - 코드 50,000줄(줄당 표준 80자)
  - 지난 5년 동안 보낸 모든 문자 메시지
  - 평균 길이의 영어 소설 8권
  - 평균 길이 200분이 넘는 팟캐스트 에피소드 스크립트

Gemini CLI: 핵심 기능과 한계

핵심 기능

- 코드 생성·수정·디버그·리팩토링을 자연어로 수행
- 파일 조작, 쉘 명령 실행, Google Search 자동 호출 내장
- MCP 서버 연동으로 외부 도구 확장 (GitHub, Slack, DB 등)
- GEMINI.md로 프로젝트별 커스텀 프롬프트 설정 가능

한계 및 주의사항

- Claude Code 등 경쟁 도구 대비 복잡한 엔지니어링 작업에서 다소 부족

Gemini CLI 환경 설정 실습

- Gemini CLI 설치 및 Google 계정 인증
- 기본 명령어 체험: 코드 생성, 파일 수정, 질의응답
- Antigravity 터미널에서 Gemini CLI 연동 확인
- 간단한 코드 생성 실습으로 동작 확인

Node.js 설치

>winget install OpenJS.NodeJS.LTS --silent

>npm --version

```
(base) C:\Users\trimu>winget install OpenJS.NodeJS.LTS --silent
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the back
rly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes  [N] No: Y
Found Node.js (LTS) [OpenJS.NodeJS.LTS] Version 24.14.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://nodejs.org/dist/v24.14.0/node-v24.14.0-x64.msi
  ■■■
```

Gemini CLI 설치

>npm install -g @google/gemini-cli

>gemini --version

```
(base) C:\Users\trimu>npm install -g @google/gemini-cli
npm warn deprecated prebuild-install@7.1.3: No longer maintained. Please
; alternatives are available.
npm warn deprecated node-domexception@1.0.0: Use your platform's native D
npm warn deprecated glob@10.5.0: Old versions of glob are not supported,
ilities, which have been fixed in the current version. Please update. Sup
bitant rates) by contacting i@izs.me

added 627 packages in 28s

175 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New minor version of npm available! 11.9.0 -> 11.11.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.11.0
npm notice To update run: npm install -g npm@11.11.0
npm notice

(base) C:\Users\trimu>gemini --version
0.31.0
```

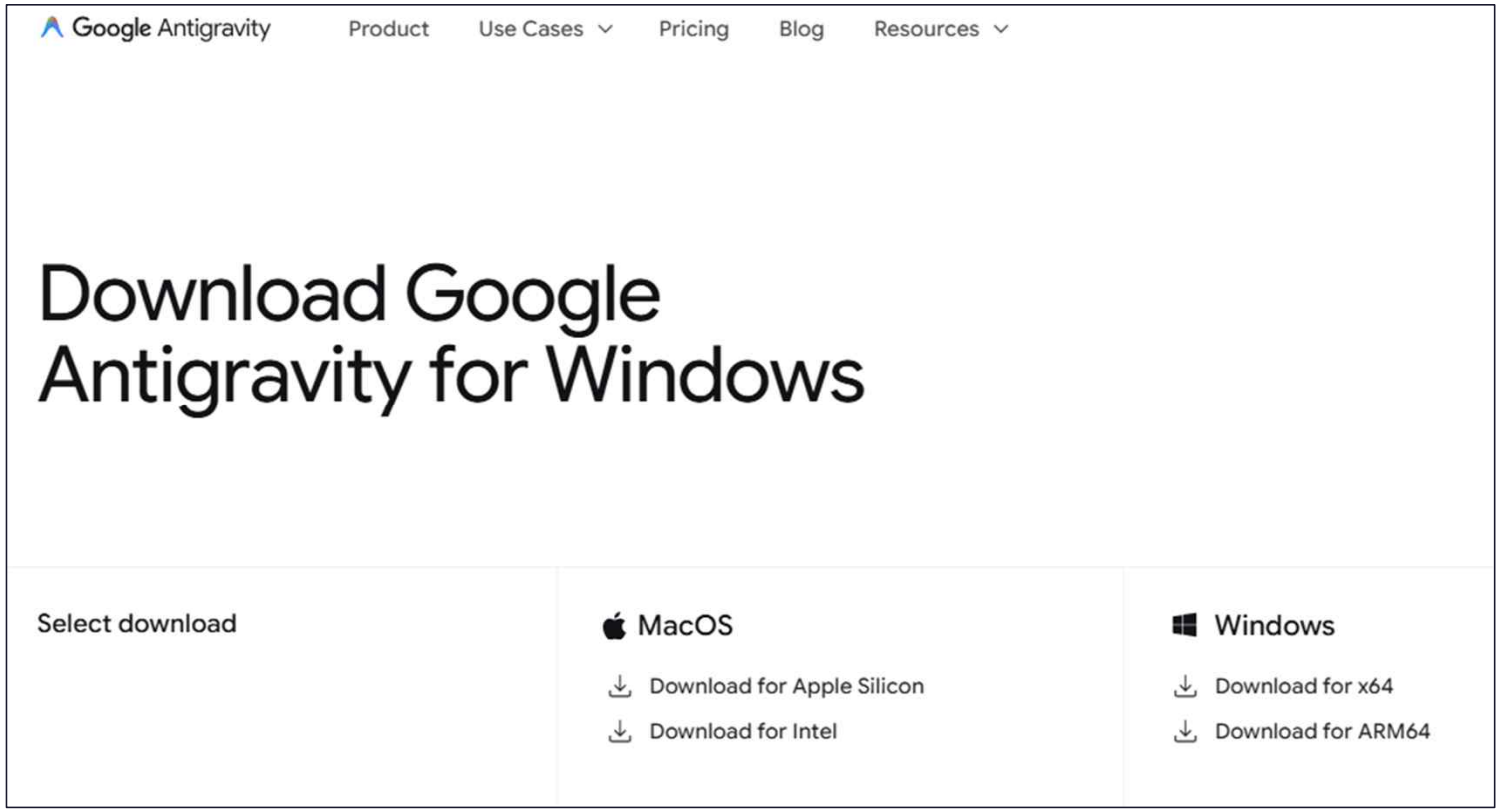
/auth

```
> /auth

i Attempting to open authentication page in your browser.
  Otherwise navigate to:
  https://accounts.google.com/o/oauth2/v2/auth?redirect_uri=http%3A%2F%2F127.0.0.1%3A53
  46%2Foauth2callback&access_type=offline&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth
  2Fcloud-platform%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email%20https%3
  %2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.profile&state=3c26b5aee285a9612f48d4da5f98
  3853847ba129649ed37278075b4aa9e4cc9&response_type=code&client_id=681255809395-oo8ft2d
  rdrnp9e3aqf6av3hmdib135j.apps.googleusercontent.com

i Waiting for authentication...

i Authentication succeeded

  You have successfully logged in with Google. Gemini CLI needs to be restarted.
  Press 'r' to restart, or 'escape' to choose a different auth method.
```

AI Code Editor 비교

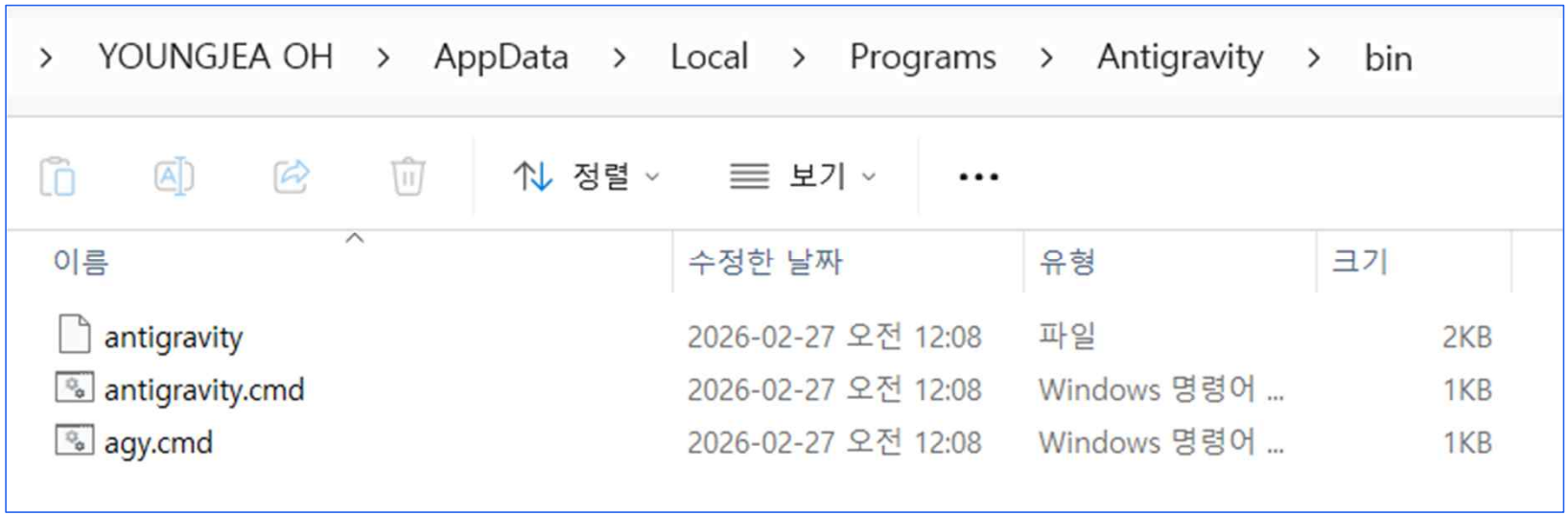
	VS Code	Cursor	Antigravity
무료 체험	완전 무료	14일 Pro 트라이얼	현재 전체 무료
유료 플랜	Copilot $10~19/월	Pro $20 / Ultra $200	미정
과금 방식	구독제	구독 + 크레딧 종량제	Rate Limit (5h 주기)
에이전트 모드	Copilot Agent (프리뷰)	Agent / Composer	Agent-First 설계
멀티 에이전트	×	Background Agent	✓ Manager View
브라우저 자동화	×	×	✓ 내장 브라우저
모델 선택	제한적	GPT, Claude, Gemini 등	Gemini, Claude, GPT-OSS
확장 / 생태계	✓ 최대 규모	✓ VS Code 호환	✓ VS Code 호환

Antigravity 설치 https://antigravity.google/download

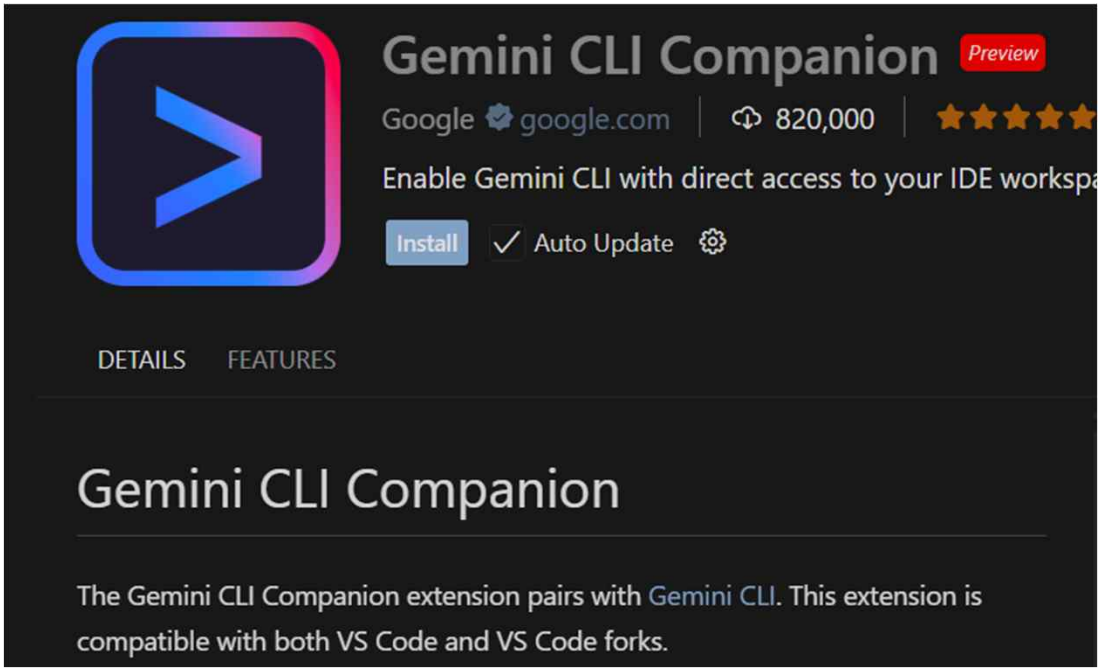


antigravity → agy 별명 등록

```
copy "C:\Users\trimu\AppData\Local\Programs\Antigravity\bin\antigravity.cmd"
     "C:\Users\trimu\AppData\Local\Programs\Antigravity\bin\agy.cmd"
```

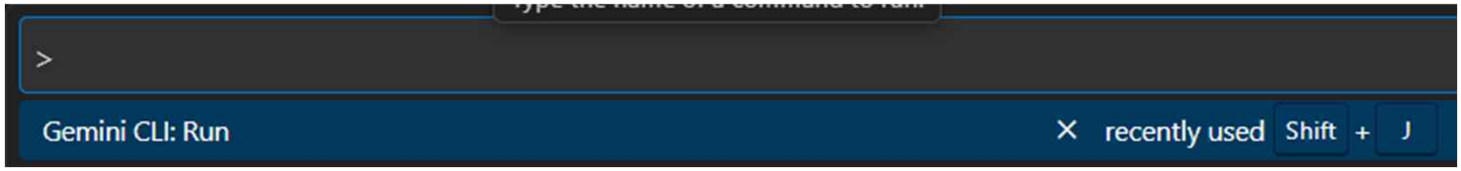


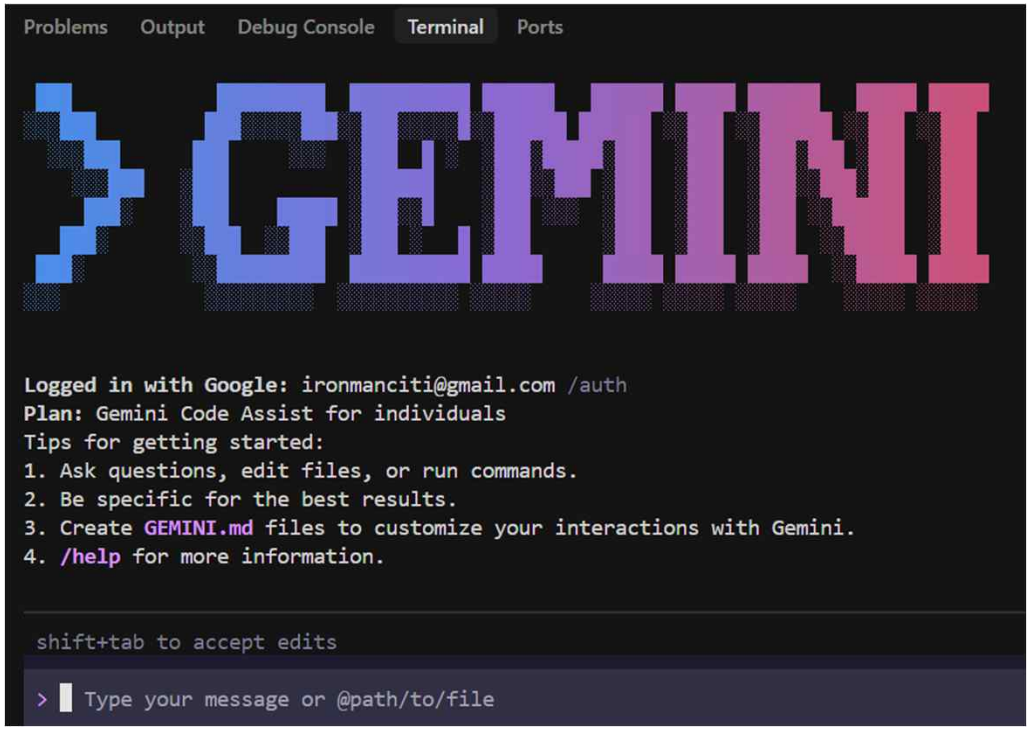
Gemini CLI extension 설치



터미널 기반 에이전트를 IDE에 통합

Gemini CLI start (ctrl + shift + P → Gemini CLI: Run)





Gemini CLI 명령어 레퍼런스 (1/4)

명령어	기능	사용 시점
/about	Gemini CLI 버전 및 시스템 정보 표시	버전 확인이나 디버깅 시
/auth	인증 상태 확인 및 계정 전환	인증 문제 발생 시
/bug	버그 리포트 생성	버그를 발견했을 때
/chat	새로운 대화 세션 시작, 저장, 복원	새 대화를 시작하거나 관리할 때
/clear	현재 대화 기록 삭제	대화 초기화가 필요할 때
/commands	사용 가능한 명령어 목록 조회 및 리로드	명령어 목록을 확인하고 싶을 때
/copy	마지막 응답을 클립보드에 복사	코드를 바로 복사하고 싶을 때
/docs	공식 문서 참조 및 검색	기능이나 사용법을 찾을 때
/directory	디렉토리 구조 표시	프로젝트 구조를 파악할 때
/ide	IDE 연동 설정 및 관리	IDE와 연동할 때

Gemini CLI 명령어 레퍼런스 (2/4)

명령어	기능	사용 시점
/init	프로젝트 초기화 및 GEMINI.md 생성	새 프로젝트 시작 시
/oncall	온콜 관련 정보 조회	온콜 상황 확인 시
/mcp	MCP 서버 연결 상태 확인 및 관리	MCP 도구 연동 확인 시
/memory	메모리 파일 관리 → .gemini\GEMINI.md	컨텍스트 설정을 관리할 때
/model	사용 중인 AI 모델 변경	다른 모델로 전환할 때
/permissions	폴더 신뢰 설정 및 권한 관리	권한 설정이 필요할 때
/policies	조직 정책 확인	정책 규정을 확인할 때
/privacy	개인정보 보호 설정 확인	프라이버시 설정 확인 시
/quit	CLI 종료	작업을 마칠 때
/resume	이전 자동 저장된 대화를 이어서 진행	이전 대화를 이어서 진행할 때

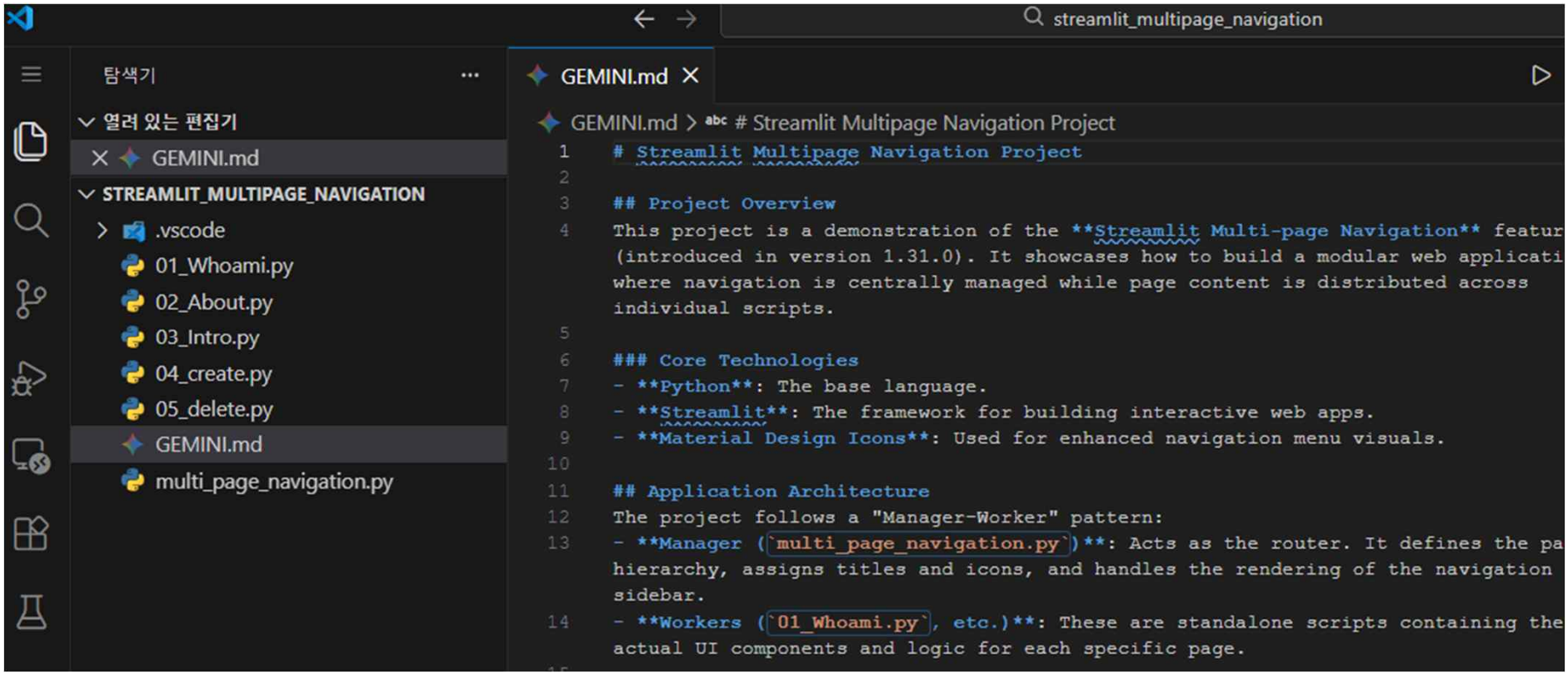
Gemini CLI 명령어 레퍼런스 (3/4)

명령어	기능	사용 시점
/stats	세션, 모델, 도구 사용 통계 조회	사용량이나 비용을 확인할 때
/tools [desc]	사용 가능한 도구 목록 및 설명 조회	도구 기능을 확인할 때
/skills	에이전트 스킬 목록 조회, 활성화, 비활성화	스킬 관리가 필요할 때
/settings	CLI 설정 확인 및 변경	설정을 조정할 때
/shells	셸 환경 설정 및 관리	셸 관련 설정이 필요할 때
/setup-github	GitHub 연동 설정	GitHub 인증 및 연동 시
/terminal-setup	터미널 테마 및 환경 설정 구성	터미널 환경을 커스터마이징할 때
/vim	Vim 모드 활성화/비활성화	Vim 키바인딩을 사용할 때

Gemini CLI 명령어 레퍼런스 (4/4)

명령어	기능	사용 시점
/extensions	확장 기능 목록 조회 및 관리 -> https://geminicli.com/extensions/	확장 기능을 설치하거나 관리할 때
/compress	필요시 수동 압축 (거의 안 쓸 것)	
/rewind	특정 메시지로 돌아가 대화 재시작	이전 응답이 잘못되어 그 시점부터 다시 진행할 때

/init - 프로젝트 초기화 및 GEMINI.md 생성



/model - 사용 중인 AI 모델 변경

```
> /model

Select Model

● 1. Auto (Gemini 3)
     Let Gemini CLI decide the best model for the task: gemini-3-pro, gemini-3-flash
  2. Auto (Gemini 2.5)
     Let Gemini CLI decide the best model for the task: gemini-2.5-pro, gemini-2.5-flash
  3. Manual
     Manually select a model
```

- **자동 라우팅:** 자동 라우팅은 먼저 프롬프트가 복잡한 작업인지 간단한 작업인지 판단합니다. 간단한 프롬프트의 경우 자동으로 Gemini 2.5 플래시를 사용합니다. 복잡한 프롬프트의 경우 Gemini 3 Pro가 활성화되어 있으면 Gemini 3 Pro를 사용하고, 그렇지 않으면 Gemini 2.5 Pro를 사용합니다.
- **프로 라우팅:** 가장 성능이 뛰어난 모델로 작업을 처리하려면 **'프로'를** `/model` 선택하십시오 . Gemini CLI는 Gemini 3 Pro가 활성화된 경우를 포함하여 사용 가능한 가장 성능이 뛰어난 모델을 우선적으로 처리합니다.

/memory - 메모리 파일 관리

명령어	기능	사용 시점
`/memory show`	현재 메모리에 로드된 **전체 내용** 표시	Gemini가 어떤 컨텍스트를 갖고 있는지 확인할 때
`/memory add`	메모리에 **새 내용 직접 추가**	규칙, 선호도 등을 즉시 기억시킬 때
`/memory refresh`	소스 파일(`GEMINI.md`)을 **다시 읽어 메모리 갱신**	`GEMINI.md`를 직접 편집한 후 반영할 때
`/memory list`	현재 로드된 **`GEMINI.md` 파일 경로 목록** 표시	어떤 파일들이 컨텍스트로 사용 중인지 확인할 때

```
> /memory add 함수 앞에 한글로 주석을 달아 주세요.

ℹ Attempting to save to memory: "함수 앞에 한글로 주석을 달아 주세요."

Action Required

?  SaveMemory in ~\.gemini\GEMINI.md

1 ## Gemini Added Memories
2 - 함수 앞에 한글로 주석을 달아 주세요.
Apply this change?

● 1. Allow once
  2. Allow for this session
  3. Modify with external editor
  4. No, suggest changes (esc)
```

```
ℹ Attempting to save to memory: "함수 앞에 한글로 주석을 달아 주세요."

✓  SaveMemory in ~\.gemini\GEMINI.md

Okay, I've remembered that: "함수 앞에 한글로 주석을 달아 주세요."
```

C: > Users > trimu > .gemini > GEMINI.md

```
1  ## Gemini Added Memories
2  - 함수 앞에 한글로 주석을 달아 주세요.
```

`/memory add`는 글로벌 파일(`~/.gemini/gemini.md`)에만 추가됩니다.

프로젝트의 GEMINI.md에 적용하려면 **직접 파일을 편집**해야 합니다:

/chat - 새로운 대화 세션 시작, 저장, 복원

명령어	기능	사용 시점
`/chat list`	저장된 대화 체크포인트 목록 표시	이전에 저장한 대화가 있는지 확인할 때
`/chat save <tag>`	현재 대화를 체크포인트로 저장	중요한 대화 흐름을 나중에 이어가고 싶을 때
`/chat resume <tag>`	저장된 체크포인트에서 대화 재개	이전에 저장한 대화를 불러와 계속할 때
`/chat delete <tag>`	저장된 체크포인트 삭제	불필요한 대화 저장본을 정리할 때
`/chat share`	현재 대화를 마크다운 또는 JSON 파일로 내보내기	대화 내용을 파일로 공유하거나 백업할 때
`/chat debug`	가장 최근 API 요청을 JSON payload로 출력	API 요청 내용을 디버깅할 때

/stats - 세션, 모델, 도구 사용 통계 조회

```
Session Stats

Interaction Summary
Session ID:                 cb0c7b89-ce41-4bc0-89cd-f410ba0d0fc3
Auth Method:                Logged in with Google (ironmanciti@gmail.com)
Tier:                       Gemini Code Assist for individuals
Tool Calls:                 1 ( ✓ 1 x 0 )
Success Rate:               100.0%
User Agreement:             100.0% (1 reviewed)

Performance
Wall Time:                  32m 4s
Agent Active:               20.6s
  » API Time:               0s (0.0%)
  » Tool Time:              20.6s (100.0%)

Auto (Gemini 3) Usage
Model                       Reqs             Usage remaining
 ────────────────────────────────────────────────────────────
gemini-2.5-flash               -     95.5% resets in 10h 20m
gemini-2.5-flash-lite          -     97.9% resets in 10h 20m
gemini-2.5-pro                 -     78.0% resets in 10h 21m
gemini-3-flash-preview         -     95.5% resets in 10h 20m
gemini-3-pro-preview           -     78.0% resets in 10h 21m
```

/settings - CLI 설정 확인 및 변경

```
> Settings

show mo

● Show Model Info In Chat (Also modified in User)                    true*
  Show the model name in the chat for each model turn.

  Show Memory Usage                                                  false
  Display memory usage information in the UI

  Show Home Directory Warning                                        true
  Show a warning when running Gemini CLI in the home directory.

  Apply To
  User Settings
● Workspace Settings
  System Settings

(Use Enter to select, Ctrl+L to reset, Tab to change focus, Esc to close)
```

```
.gemini > { } settings.json > ...
1  {
2    "ui": {
3      "showModelInfoInChat":
       true
4    }
5  }
```

```
> Type your message or @path/to/file
~\...\sampleWebsite (master*)        no sandbox        /model Auto (Gemini 3)
```

```
● Hide Context Window Percentage (Also modified in User)     false*
  Hides the context window remaining percentage.

▼

  Apply To
  User Settings
● Workspace Settings
  System Settings
```

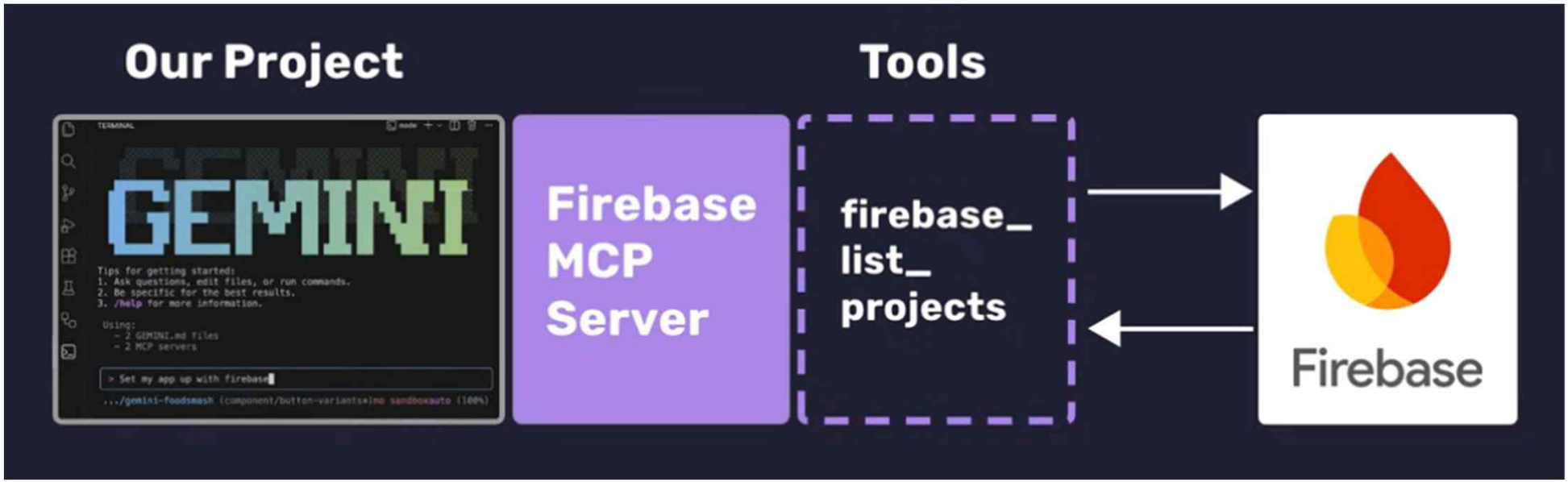
```
> Type your message or @path/to/file

~\...\sampleWebsite (master*)  no sandbox   /model Auto (Gemini 3) 100%
```

Shell Command - !

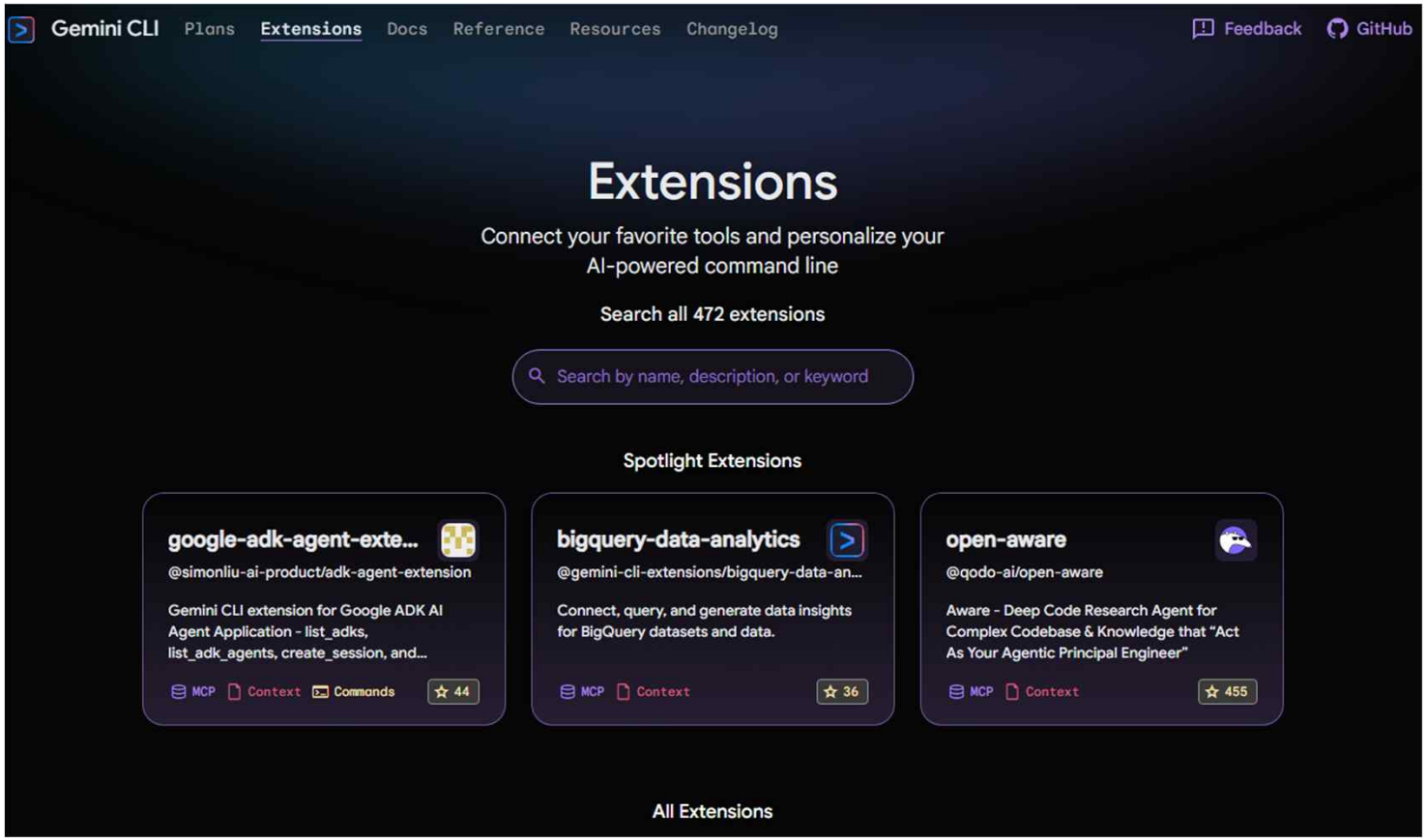
- Shell command 결과를 context 에 포함

```
$ dir

✓  Shell Command dir

... 9 hidden (Ctrl+O) ...
dar--l      2026-03-09  ♦
                         4:40                static
dar--l      2026-03-10  ♦♦
                         5:32                 templates
-a---l      2026-03-10  ♦♦
                         5:32            135 .gitignore
-a---l      2023-10-03  ♦
                         6:02              0 file.log
-a---l      2026-03-10  ♦♦
                         5:12           1267 flask_app.py
-a---l      2026-03-10  ♦♦
                         5:32           2223 GEMINI.md
-a---l      2025-12-07  ♦
                         1:26             94 requirements.txt

                                                  ? for shortcuts
shell mode enabled (esc to disable)                2 GEMINI.md files
!   Type your shell command
```

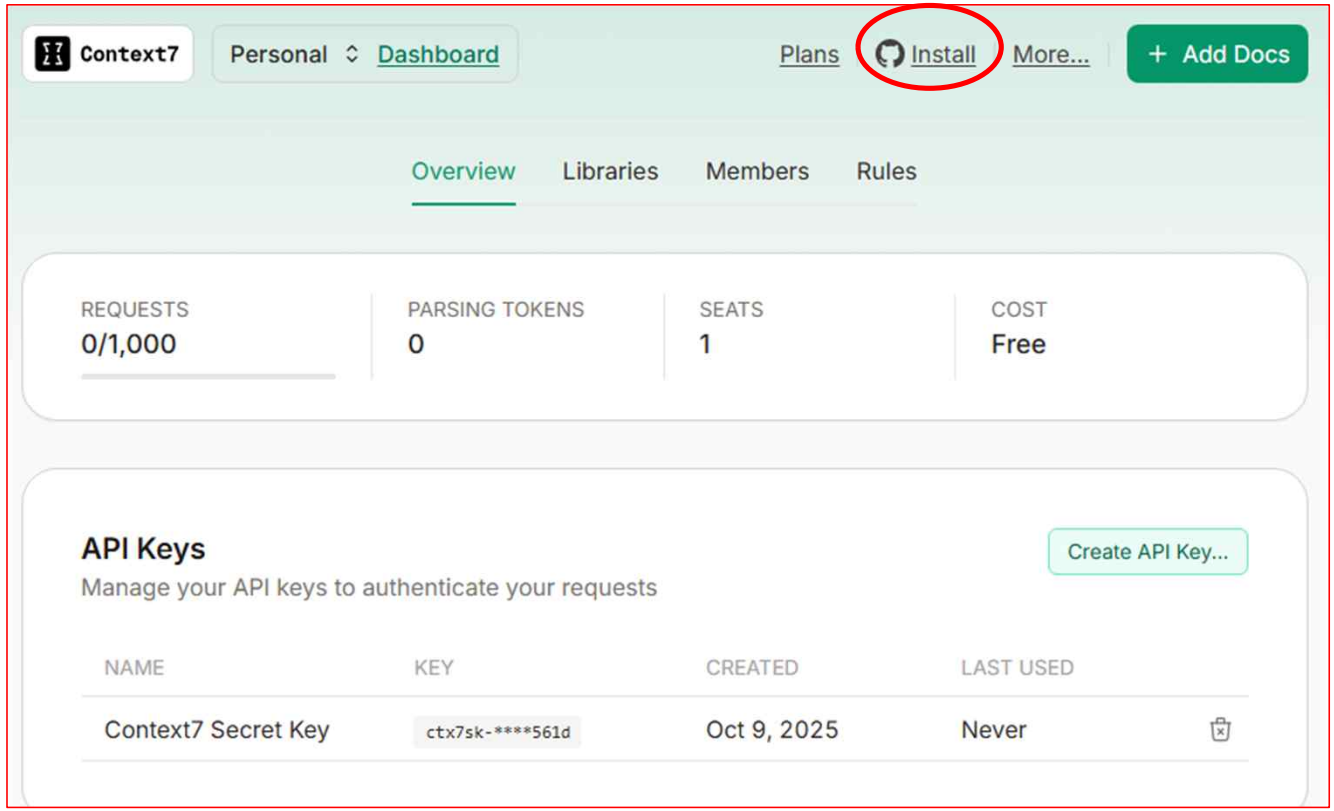
MCP Servers & Extensions



Extensions - https://geminicli.com/extensions/



Context7 API Key - https://context7.com/dashboard



Enterprise

Overview

On-Premise

Security

Clients

Claude Code

Cursor

OpenCode

All MCP Clients

CLI

SDKs

TypeScript

Agentic Tools

Overview

Vercel AI SDK

API Reference

Search

Gemini CLI

See **Gemini CLI Configuration** for details.

1. Open the Gemini CLI settings file at `~/.gemini/settings.json`
2. Add the following to the `mcpServers` object:

```
{
  "mcpServers": {
    "context7": {
      "httpUrl": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "YOUR_API_KEY",
        "Accept": "application/json, text/event-stream"
      }
    }
  }
}
```

Or, for a local server:

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp", "--api-key", "YOUR_API_KEY"]
    }
  }
}
```

settings.json ● .env ●

.env

```
CONTEXT7_API_KEY=ctx7sk-***************-1a05867fa769
```

.gemini > settings.json > ...

```
...
{
  "ui": {
    "showModelInfoInChat": true,
    "footer": {
      "hideContextPercentage": false
    }
  },
  "mcpServers": {
    "context7": {
      "httpUrl": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "${CONTEXT7_API_KEY}",
        "Accept": "application/json, text/event-stream"
      }
    }
  }
}
```

```
> /mcp

Configured MCP servers:

● context7 - Ready (2 tools)
  Tools:
  - query-docs
  - resolve-library-id
```

AI 활용 소프트웨어 개발 사이클

STEP 1
PRD:
무엇을 만들
것인가

→

STEP 2
소프트웨어
아키텍처:
어떤 기술로
어떻게 구성

→

STEP 3
DB 스키마
API/UI 설계:
데이터 모델
인터페이스

→

STEP 4
Task
Decomposition:

작업 단위 분해

→

STEP 5
코드 생성:
태스크 단위
구현

→

STEP 6
테스트 / 리뷰
품질 검증
피드백

⚠ **가장 빠지기 쉬운 단계: DB 스키마 / API / UI 설계**
이 단계를 건너뛰면 AI가 태스크마다 다른 테이블 구조/엔드포인트를 생성
→ **나중에 통합 불가!**

피드백 루프: 테스트 결과 → PRD 재검토 → 아키텍처 수정 → 반복 개선

AI에게 효과적인 프롬프트 작성법

① 역할 부여 (Role)

요청 전 AI에게 역할을 명시
"Node.js 백엔드 시니어 개발자로서 응답해줘"

② 맥락 제공 (Context)

프로젝트 배경, 기술 스택, 기존 코드 구조 등 상황 설명
"Express + PostgreSQL 기반 REST API 프로젝트에서..."

③ 구체적 지시 (Specific)

모호한 요청 대신 입력/출력/예외 명시
"상품명, 가격, 재고를 받아 JSON으로 반환하는 API 작성"

④ 제약 조건 (Constraints)

기술적 제약, 성능 요구, 보안 요구사항 명시
"응답 2초 이내, JWT 인증, SQL Injection 방지 필수"

⑤ 출력 형식 (Format)

원하는 결과물의 형태를 명확히 지정
"TypeScript로 작성, ESLint 적용, 주석 포함, 테스트 포함"

핵심 원칙

모호함 → 구체성
프롬프트가 구체적일수록
AI 출력 품질이 향상된다

AI 기반 PRD 작성 및 요구사항 정의

PRD(Product Requirements Document**)란?**

- 제품의 무엇을, 왜, 어떻게 구현할지 정의하는 핵심 문서 — AI로 속도·일관성·완성도 향상

AI 기여 영역

- **요구사항 도출·구조화** 비정형 데이터 → **기능 요구사항 vs 비기능 요구사항** 자동 분류
- **PRD 템플릿 자동 생성** 프롬프트 한 번으로 표준 섹션(개요·목표·페르소나·AC 등) 초안 완성
- **유저 스토리·시나리오 작성** 대량 유저 스토리 생성 + 엣지 케이스 도출
- **일관성 검증·갭 분석** 요구사항 간 충돌·누락·모호 표현 자동 탐지
- **우선순위 추천** 예) MoSCoW / RICE 기반 자동 제안

기능 요구사항 vs 비기능 요구사항

구분	기능 요구사항 (Functional)	비기능 요구사항 (Non-Functional)
정의	시스템이 수행해야 할 구체적인 동작과 기능	시스템의 품질 속성과 운영 조건에 대한 제약
핵심 질문	"시스템이 무엇을 하는가?"	"시스템이 얼마나 잘 하는가?"
예시	• 사용자 로그인 기능 • 상품 검색 및 필터링 • 주문 내역 조회 API	• 응답 시간 2초 이내 • 동시 접속 1,000명 지원 • 99.9% 가용성 보장
AI 활용	AI가 직접 구현 코드 생성 테스트 케이스 자동 작성	성능 테스트 스크립트 생성 모니터링 설정 코드 생성
PRD 작성 팁	입력/출력/예외 상황을 구체적으로 명시	수치화된 기준 제시 (응답 시간, 처리량 등)

RICE 우선순위 프레임워크

제품/프로젝트 기능의 우선순위를 객관적으로 점수화하는 의사결정 도구

RICE Score = (**Reach** × **Impact** × **Confidence**) / **Effort**

Reach
도달 범위
일정 기간 내 영향받는
사용자/고객 수

예: 분기당 5,000명

Impact
영향도
고객당 영향 정도
(0.25 ~ 3 스케일)

3=대규모 / 1=중간 / 0.25=최소

Confidence
확신도
추정치에 대한
확신 수준 (%)

100%=높음 / 80%=중간 / 50%=낮음

Effort
투입 노력
필요한 인력과 시간
(인월, man-months)

예: 3 M/M

계산 예시

추천 알고리즘 도입: **(5,000 × 2 × 0.8) / 3 = 2,667점** ← 다른 후보 기능과 비교하여 높은 순으로 우선 개발

Bad PRD vs Good PRD

❌ Bad PRD

"상품 관리 시스템 만들어줘.
로그인도 필요하고
CRUD 기능 구현해줘."

문제점:
- 대상 사용자/문제 정의 없음
- 기술 스택/제약 조건 미명시
- 인수 조건/성공 기준 부재

✅ Good PRD

"Node.js + Express + PostgreSQL로
상품 관리 REST API 구현.
JWT 인증, 입력값 검증(Joi),
에러처리, 응답 2초 이내,
TypeScript로 작성."

강점:
- 기술 스택 명시 → 정확한 코드 생성
- 비기능 요구 포함 → 품질 보장

핵심: PRD에 30분 투자하면 AI가 생성하는 코드의 완성도가 극적으로 향상됩니다

PRD 작성 실습 - 사내 식당 메뉴 추천 앱

Step 1 프롬프트
"직원 500명의 점심 메뉴 선택 문제. 알레르기 관리, 잔반 감소가 목표. Functional / Non-functional 요구사항으로 분류해 줘."

Step 2 PRD 템플릿 자동 생성
"위 요구사항을 기반으로 PRD를 작성해 줘. 포함 섹션: 제품 개요, 목표, 타겟 페르소나, 유저 스토리, Acceptance Criteria, 제약사항 포함"

Step 3 유저 스토리량시나리오 작성
"이 앱의 유저 스토리를 10개 작성하고, 각각에 대해 엣지 케이스 2개씩 도출해 줘."

Step 4 일관성 검증량갭 분석
"위 PRD에서 요구사항 간 충돌이나 누락, 모호한 표현이 있는지 검토해 줘."

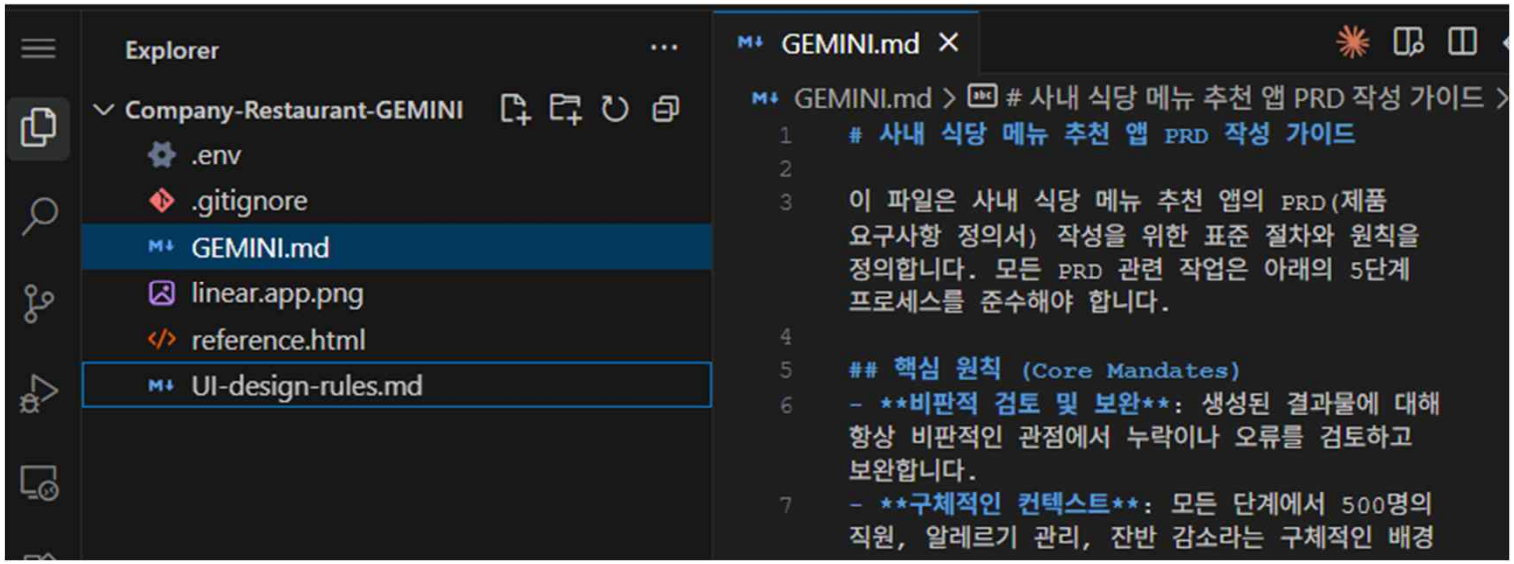
Step 5 우선순위 추천 (RICE)
"위 기능 목록을 RICE 프레임워크로 평가해 줘. 사용자 500명, 개발팀 3명, 일정 2개월 기준."

비판적 검토·보완 · 컨텍스트를 구체적으로 · 프롬프트 반복 비교 · 도메인 전문가 최종 검토 필수

- **PRD 정의를 위한 Prompt 파일 작성**

```
> [Image ...3530727254.png] 이 내용으로 PRD 작성을 할 수 있도록
  GEMINI.md 파일을 만들어줘
```

GEMINI.md 에 저장



소프트웨어 아키텍쳐 : 기술 제약조건 - 사내 식당 메뉴 추천 앱

Tech Stack

Frontend
HTML + CSS + JS (프레임워크 없음)
Backend
Flask (Python) + Jinja2 템플릿
DB
SQLite (단일 파일 app.db)
인증 / 배포
없음 (학습용으로 생략, flask run)

아키텍처

Flask 모놀리식 (MVC 패턴)
REST API 기반 통신

데이터 소스

알레르기 정보
allergen_profiles 테이블 (사전 등록)
메뉴 데이터
menus 테이블 (영양사 직접 입력)
식사 선택 이력
meal_selections 테이블

프로젝트 제약

사용자 500명 | 개발팀 3명 | 2개월
보존기간 제한 없음 (학습용)
핵심 목표: 알레르기 관리, 잔반 감소

- PRD 에 추가하기 위한 기술 제약 조건 Prompt 파일 작성

```
> [Image ...3531213281.png] 이 소프트웨어 아키텍쳐 슬라이드를
  참고해서 @GEMINI.md 에 기술 제양 조건을 업데이트해줘
```

GEMINI.md 업데이트

```
GEMINI.md ↔ Modified ×
# 사내 식당 메뉴 추천 앱 PRD 작성 가이드
## 핵심 원칙 (Core Mandates)
- **도메인 전문가 검토**: 최종 결과물은 도메인 전문가의 검토를 위한 형태로 정리되어야 합

## 기술 제약 조건 (Technical Constraints)
- **Tech Stack**:
  - **Frontend**: HTML + CSS + JS (프레임워크 사용 안 함)
  - **Backend**: Flask (Python) + Jinja2 템플릿
  - **Database**: SQLite (단일 파일 `app.db`)
  - **인증/배포**: 별도 인증 없음, `flask run` 로컬 실행 (학습용)
- **Architecture**:
  - Flask 모놀리식 구조 (MVC 패턴 적용)
  - REST API 기반 통신
- **Data Schema**:
  - `allergen_profiles`: 알레르기 정보 (사전 등록)
```

DB 스키마 설계 - 사내 식당 메뉴 추천 앱

프론프트 예시

현재까지 작성된 PRD와 기술 제약조건을 기반으로 SQLite DB 스키마를 설계해줘.

요구사항:

- CREATE TABLE SQL문으로 작성
- 테이블 간 관계(FK) 명시
- 각 컬럼에 한글 주석 포함

추가 요구사항

- 테스트용 INSERT 샘플 데이터 5건씩 포함
- ERD를 Mermaid 다이어그램으로 그려줘

예상 테이블

`employees`
직원 정보 및 알레르기 프로필

`allergen_profiles`
직원별 알레르기 정보

`menus` / `meal_selections`
메뉴 데이터 및 식사 선택 이력

PRD + 기술 제약조건을 함께 제공해야 AI가 일관된 스키마를 설계합니다

- DB 스키마 설계를 위한 Prompt 파일 작성

```
> [Image ...3531508125.png] 이 슬라이드를 이용해서 DB 스키마
  문서를 작성해서 @GEMINI.md 에 업데이트해줘
```

GEMINI.md 업데이트

GEMINI.md

식당 메뉴 추천 앱 PRD 작성 가이드 > ## PRD 및 설계 프로세스 > ### Step 6. DB

```md
# 사내 식당 메뉴 추천 앱 PRD 작성 가이드

## DB 스키마 설계 지침 (Database Schema Design)
PRD와 기술 제약 조건을 기반으로 SQLite DB 스키마를 설계할 때 다음 사항을 준수합니다.
- **SQL 작성**: 표준 `CREATE TABLE` 문을 사용합니다.
- **관계 명시**: 테이블 간의 외래 키(FK) 관계를 명확히 정의합니다.
- **주석 포함**: 모든 컬럼에 대해 한글로 용도 및 특성 주석을 추가합니다
- **ERD 작성**: Mermaid 다이어그램을 사용하여 테이블 간의 관계를 시각화합니다.
- **테스트 데이터**: 각 테이블당 최소 5건의 샘플 `INSERT` 문을 포함합니다.
- **필수 테이블**:
  - `employees`: 직원 기본 정보
  - `allergen_profiles`: 직원별 알레르기 유발 물질 정보
  - `menus`: 식단 메뉴 데이터 (영양사 입력)
  - `meal_selections`: 직원의 식사 선택 및 잔반 관련 이력
```

UI 설계 - 사내 식당 메뉴 추천 앱

프롬프트 예시

현재까지 작성된 PRD와 기술 제약조건을 기반으로
주요 화면의 UI를 설계해줘.

요구사항:
- 화면별 와이어프레임 (레이아웃 구조)
- 각 화면의 구성요소 목록
- 화면 ↔ API 매핑표
- 컴포넌트 트리 구조 (React 기준)
- 공통 UI 패턴/디자인 시스템 정의

추가 요구사항

- 반응형 디자인 (모바일/데스크톱)
- Tailwind CSS 기반 스타일 가이드
- 네비게이션 흐름도 (Mermaid)

예상 화면 목록

로그인 / 프로필 설정
알레르기 정보 입력

오늘의 메뉴 조회
메뉴 목록 + 알레르기 경고

메뉴 추천 결과
개인별 맞춤 추천 표시

식사 선택 / 피드백
선택 기록 + 평점 입력

DB 스키마 · API 설계와 동시에 진행해야 화면↔엔드포인트 정합성이 유지됩니다

현재 사람들이 디자인하는 세 가지 주요 방식

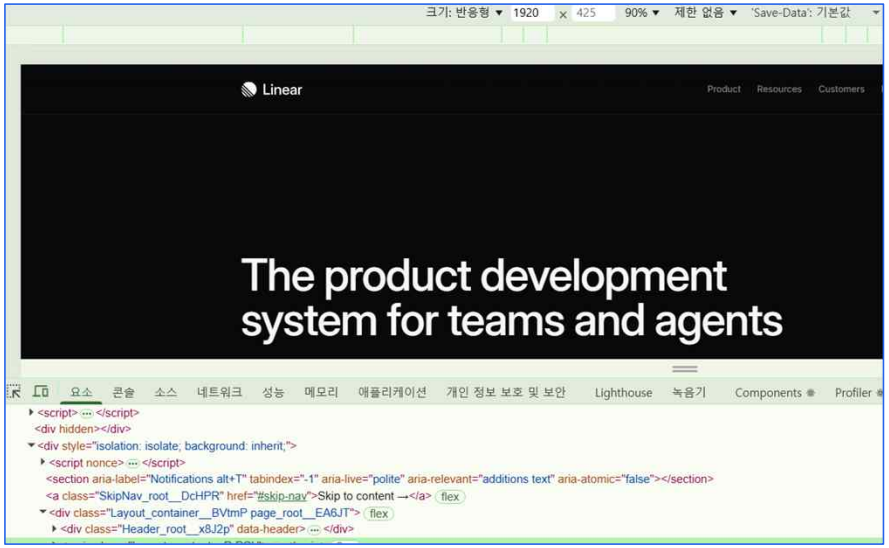
구분	디자인 + 스크린샷 루프	음성 녹취록 + 편집	컴포넌트 활용
입력 방식	스크린샷/목업 이미지 전달	음성으로 요구사항 설명	기존 컴포넌트 라이브러리 참조
정확도	높음 (시각적 참조 기반)	중간 (해석 필요)	컴포넌트 품질에 의존
속도	중간 (반복 수정 필요)	빠름 (실시간 대화)	빠른 프로토타입 제작
장점	정확한 시각적 결과물	자연스러운 의사소통	일관된 디자인 시스템
단점	초기 디자인 필요	세밀한 조정 어려움	컴포넌트 제한에 종속
적합 상황	정밀한 UI 구현 시	빠른 아이디어 전달 시	기존 시스템 확장 시

Claude Code 에 참조 Website 인식 시키기

1. 실제 서비스 UI 패턴 찾기(ex. mobbin.com)
2. curl -o reference.html https://linear.app/
3. Image capture & Image size 줄이기(context size 제한 회피)
4. UI-design-rules.md 생성

image capture

검사 (ctr+shift+i)



Ctr+shift+P

Image size 줄이기

https://www.iloveimg.com/ko/resize-image/resize-png

UI-design-rules.md 생성

```
> @reference.html [Image linear.app.png] 을 이용해서 사용자 화면 디자인 가이드라인을 작성하여
  UI-design-rule.md 파일을 생성해줘.
```

GEMINI.md 업데이트

```
> @UI-design-rules.md 를 이용해서 UI 지침을 @GEMINI.md 에 업데이트해줘
```

```
M↓ GEMINI.md > # 사내 식당 메뉴 추천 앱 PRD 작성 가이드 > ## PRD 및 설계 프로세
 1   # 사내 식당 메뉴 추천 앱 PRD 작성 가이드
20   ## DB 스키마 설계 지침 (Database Schema Design)
27
28   ## UI 디자인 지침 (UI Design Guidelines)
29   Linear.app의 디자인 언어를 계승하여 현대적인 Dark Mode UI를
     지향합니다.
30   - **디자인 원칙**: Extreme Clarity, Modern Dark Aesthetic,
     Interactive Feedback.
31   - **컬러 팔레트**:
32     - 배경: `#08090A` / 패널: `#111214`
33     - 텍스트: Primary `#F7F8F8`, Secondary `#9C9DA1`, Accent
       `#5E6AD2`
34     - 상태: 경고(Red) `#FF4D4D`, 성공(Green) `#4DFFAD`
35   - **컴포넌트 규격**:
36     - Border: `1px solid rgba(255, 255, 255, 0.08)`
```

GEMINI.md 의 context 포함 확인

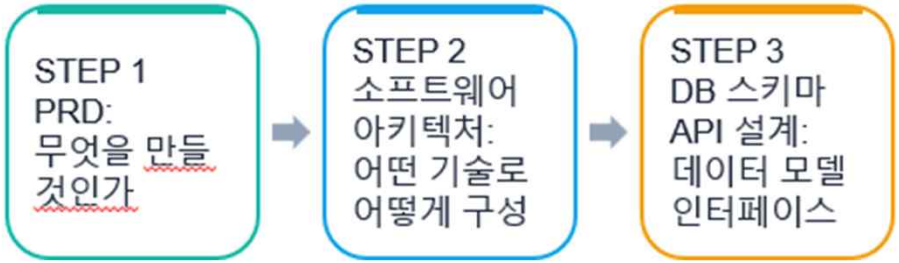
```
> /memory show
```

```
Current memory content from 4 file(s):
---
--- Global ---
--- Context from: ..\..\..\.gemini\gemini.md ---
```

```
--- Context from: gemini.md ---
# 사내 식당 메뉴 추천 앱 PRD 작성 가이드

이 파일은 사내 식당 메뉴 추천 앱의 PRD(제품 요구사항 정의서) 작성을 위한 표준 절차와 원칙을
정의합니다. 모든 PRD 관련 작업은 아래의 5단계 프로세스를 준수해야 합니다.

## 핵심 원칙 (Core Mandates)
- 비판적 검토 및 보완: 생성된 결과물에 대해 항상 비판적인 관점에서 누락이나 오류를 검토하고
보완합니다.
- 구체적인 컨텍스트: 모든 단계에서 500명의 직원, 알레르기 관리, 잔반 감소라는 구체적인 배경
유지합니다.
```

```
### Step 7. UI 디자인 프로토타이핑 (NEW)
- 작업: UI 디자인 지침을 준수하여 주요 화면의 HTML/CSS 구조 설계.

---
이 가이드는 프로젝트의 초기 단계부터 완료까지 일관된 품질을 유지하기 위해 사용됩니다.
--- End of Context from: gemini.md ---
```

- Task Decomposition



STEP1~3 문서 완성

→ STEP4에서 작업을 적절한 크기로 분해

→ 각 작업 단위마다 Plan Mode

→ 해당 작업 코드 생성/수정

→ 테스트/리뷰

예)
- 인증
- 사용자 관리
- 메뉴 추천
- 알림
- 관리자 통계

- **복잡한 Project – Task 별로 Plan Mode 적용**

- **간단한 Project – 전체적으로 Plan Mode 적용**

AI Pair Programming 사이클

1. Plan (계획)

- 요구사항 정리
- AI에게 목표 설명
- 입력/출력/제약 명시

→

2. Code (코드 생성)

- AI가 코드 생성
- 개발자가 실행 확인
- 동작 여부 테스트

→

3. Review (검토)

- 코드 품질 확인
- 보안 취약점 점검
- 에러 처리 검증

→

4. Refine (개선)

- 피드백 반영
- 리팩토링 요청
- 테스트 추가

↻ **반복 사이클**: Refine 후 다음 Task의 Plan으로 이동 — 함수 단위로 반복하면 품질과 속도 모두 확보

```
> 현재까지 작성된 PRD 를 기반으로 시스템 구현 계획을 세워줘.
```

```
Ready to start implementation?

시스템 구현 계획 (System Implementation Plan)

1. Background & Motivation (배경 및 동기)
사내 식당 메뉴 추천 앱은 직원(약 500명)들의 매일 점심 메뉴 선택에 대한 고민을 해결하고, 개인의
알레르기 정보를 관리하며, 수요 예측을 통해 잔반을 줄이는 것을 목표로 합니다.
현재까지 정리된 GEMINI.md와 UI-design-rules.md의 PRD 및 디자인 가이드라인을 기반으로, 모놀리식
아키텍처의 웹 애플리케이션으로 시스템을 구현하고자 합니다.

2. Scope & Impact (범위 및 영향)
 - 대상 사용자: 사내 직원 500명
 - 주요 기능:
   - 오늘의 메뉴 목록 조회 및 추천
   - 사용자 알레르기 설정 및 알레르기 유발 메뉴 경고
   - 점심 메뉴 선택 (수요 예측용 데이터 수집)
... last 176 lines hidden (Ctrl+O to show) ...

● 1.  Yes, automatically accept edits
      Approves plan and allows tools to run automatically
  2.  Yes, manually accept edits
      Approves plan but requires confirmation for each tool
```

테스트 및 문서 생성

- 구현된 코드에 대한 단위 테스트 자동 생성
- 테스트 실행 및 실패 케이스 AI로 수정
- README.md 자동 생성 (설치 가이드, 사용법, 아키텍처 설명)
- API 문서 자동 생성
- 코드 주석 및 docstring 일괄 추가

AI 활용 테스트 생성

테스트 생성 프롬프트 템플릿

```
// 프롬프트 예시
[createUser 함수]에 대한 테스트를 작성해줘.

포함할 케이스:
  1. 정상 입력으로 사용자 생성
  2. 이메일 중복 시 에러 발생
  3. 빈 이름 입력 시 검증 실패
  4. DB 연결 실패 시 에러 처리
```

테스트 전략 3단계

Step 1
AI에게 테스트 먼저 작성 요청

Step 2
테스트 통과하는 구현 코드 생성

Step 3
테스트 커버리지 확인 및 보완

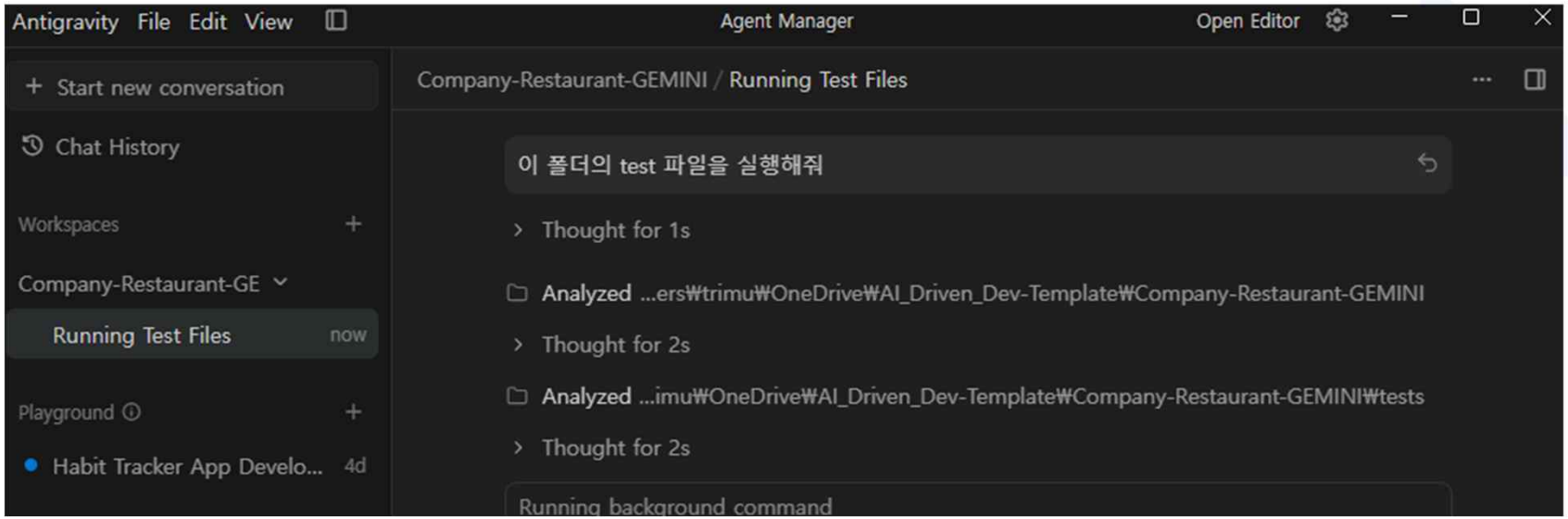
```
> [Image ...3537444121.png] 이 슬라이드와 playwright 를 이용한 테스트 케이스를
  작성해줘. 작성한 테스트 케이스는  test 폴더에 저장해줘.
```

1. 라이브러리 설치
   pip install playwright pytest-playwright

2. 브라우저 엔진 설치 (필수)
   playwright install chromium

```
슬라이드에서 제안한 4가지 핵심 케이스를 모두 포함하는 통합 UI 테스트를
tests/test_ai_generated.py에 저장했습니다.


다음 명령어로 테스트를 실행하여 결과를 확인하실 수 있습니다:

 1 pytest tests/test_ai_generated.py


테스트 요약
 * Case 1 (정상 입력): 실제 EMP001 사번으로 로그인하여 메뉴 선택이 성공하는지
   확인합니다.
 * Case 2 (중복 선택): 동일 메뉴를 반복 클릭할 때 UI 흐름이 깨지지 않는지
   확인합니다.
 * Case 3 (검증 실패): 사번을 비워두었을 때 사용자에게 경고 알림창이 뜨는지
   확인합니다.
 * Case 4 (에러 처리): Playwright의 Route 기능을 사용하여 서버 에러(500) 상황을
   강제로 만들고, 사용자가 "실패" 메시지를 적절히 받는지 검증합니다.
```

Agent Manager



`tests` 폴더 내에 있는 API 테스트 파일과 UI 테스트 파일을 모두 실행완료 했습니다.

두 테스트 파일 모두 모든 항목이 **성공(Passed)**했습니다!

진행한 테스트 내역과 결과입니다:

1. 백엔드/API 테스트 (tests/test_app.py)

> Flask의 Test Client를 사용해 앱 내부 API의 정상 동작 여부를 확인했습니다.

- **결과:** `5 passed` (모두 성공)
- **테스트 항목:**
  1. 직원 정보 조회 (정상)
  2. 존재하지 않는 사번 조회 (에러)
  3. 메뉴 선택 정보 정상 저장
  4. 필수 데이터 누락 시 검증 (에러)
  5. 주간 식단표 데이터 조회

기술 문서 자동화

README.md

- 프로젝트 개요 및 목적
- 설치 및 실행 방법
- 환경 변수 설정 안내
- 폴더 구조 설명
- 기여 가이드라인

API 문서

- 엔드포인트 목록 및 설명
- 요청/응답 스키마 정의
- HTTP 상태 코드 및 에러 처리
- 인증 방식 설명
- 사용 예제 (curl/fetch)

인라인 주석

- 함수/메서드 목적 설명
- 매개변수 타입 및 제약 설명
- 반환값 타입 및 예외 사항

AI 활용 팁: 코드 완성 직후 문서 생성을 요청하면 코드와 문서의 일관성이 높아짐

문서 자동화 프롬프트와 품질 관리

문서 자동화 프롬프트 예시

```
// README 생성
"이 프로젝트의 README.md를 작성해줘.
설치, 실행, 환경변수, 폴더구조 포함"

// API 문서 생성
"이 Express 라우터의 API 문서를
OpenAPI 3.0 형식으로 작성해줘"

// 인라인 주석 추가
"이 파일의 모든 함수에 JSDoc 주석을
추가해줘. 파라미터 타입과 반환값 포함"
```

문서 품질 관리

① 코드 변경 시 문서 동시 업데이트
- " 코드와 문서를 함께 수정해줘"

② 출력 형식 명시
- Markdown, OpenAPI, JSDoc 등

③ 본문 검증 필수
- AI가 잘못된 파라미터나 가짜 예제를 만들 수 있음

④ 팀 컨벤션 준수
- CLAUDE.md에 문서 스타일 규칙 명시

핵심: 문서는 코드와 함께 생성·업데이트 — 별도 작업이 아닌 AI 워크플로우의 일부로 통합

채팅 Cowork 코드

image.png

현재의 코드 베이스를 이용해서 슬라이드에 언급된 문서들을 작성해줘

나머지 소스 파일들을 읽어서 전체 코드베이스를 파악하겠습니다.

Read and analyzed Flask application test file

이제 전체 코드베이스를 파악했습니다. 슬라이드에 명시된 3가지 문서를 모두 작성하겠습니다.

---

1. README.md 작성

Created comprehensive README documentation for restaurant menu recommendation system

2. API 문서 작성 (OpenAPI 3.0 형식)

Created API documentation for restaurant menu recommendation system

3. 인라인 주석 (Docstring) 추가

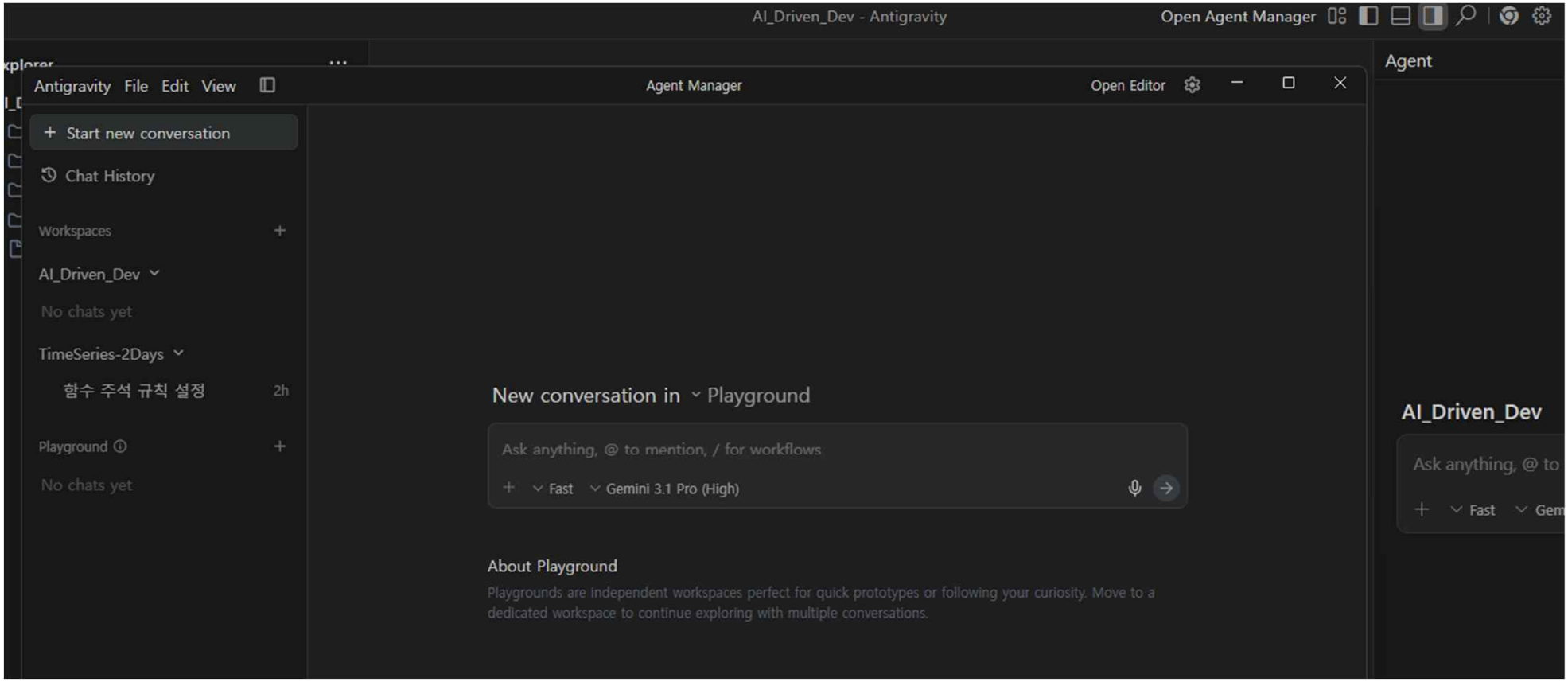
이제 `app.py`, `models/db_manager.py`, `static/js/main.js` 에 상세한 인라인 주석을 추가합니다.

감사합니다

부록

Antigravity Agent Manager 소개

Agent Manager



Playground / Habit Tracker Planning

아주 간단한 습관 추적 앱을 만들어 주세요.
사용자가 습관을 추가하고, 오늘 완료로 표시할 수 있어야 하며, 모든 데이터는 localStorage를 사용하여 저장되어야 합니다.
디자인은 깔끔하고 미니멀하며, 모바일 친화적이어야 하고, 불필요한 추가 기능은 없어야 합니다.
Implementation Plan 문서를 작성해 주세요

Thought for 16s

Planning Habit Tracking App
Planning the creation of a minimalist habit tracking web application.

Files Edited
implementation_plan.md task.md

Progress Updates Expand all
1 Creating task.md and implementation_plan.md

작성된 Implementation Plan(구현 계획)을 확인해주세요. 미니멀하고 모바일 친화적인 디자인, 로컬 스토리지 기반의 간단한 습관 추적 앱에 대한 구조와 로직(함수 한글 주석 포함)을 계획했습니다. 확인 후 승인해주시면 바로 코딩을 시작하겠습니다!

implementation_plan.md Open Proceed

Copy

Ask anything, @ to mention, / for workflows

Planning Gemini 3.1 Pro (High)

Implementation Plan now Review Proceed

Habit Tracker Implementation Plan

A very simple, minimalist habit tracker application. Users can add new habits and toggle their completion status for the current day. The data is persisted using `localStorage`.

User Review Required

Please review the proposed plan to ensure it meets your expectations for a simple, minimalist, and mobile-friendly UI, and confirms the functional requirements.

Proposed Changes

Web Application UI

[MODIFY] index.html

- Add the main document structure.
- Include a header section.
- Include an input field and a button to add new habits.
- Include a list container (`<ul id="habit-list">`) to render the habits.

[MODIFY] style.css

- Implement a minimalist and mobile-friendly design.
- Use a soft, clean color palette (e.g., modern whites/grays and subtle accent colors).
- Ensure input forms and buttons are appropriately sized for mobile taps.
- Provide clear visual indicators for completed habits (e.g., strikethrough, checkmarks).

Web Application Logic

[MODIFY] script.js

- Implement logic to load habits from `localStorage`.
- Implement adding a new habit to the state and re-rendering the list.
- Implement checking/unchecking a habit for "today".
- Use Korean comments before every function as requested by the user rule (`// 함수 앞에 한글로 주석을 달아 주세요.`).

Verification Plan

Manual Verification

- Open `index.html` in a web browser.
- Add multiple habits.
- Mark some as completed for today.

아주 간단한 습관 추적 앱을 만들어 주세요.
사용자가 습관을 추가하고, 오늘 완료로 표시할 수 있어야 하며, 모든 데이터는 localStorage를 사용하여 저장되어야 합니다.
디자인은 깔끔하고 미니멀하며, 모바일 친화적이어야 하고, 불필요한 추가 기능은 없어야 합니다.
Implementation Plan 문서를 작성해 주세요

Thought for 16s

Planning Habit Tracking App

Planning the creation of a minimalist habit tracking web application.

Files Edited

implementation_plan.md task.md

Progress Updates Expand all

1 Creating task.md and implementation_plan.md

작성된 Implementation Plan(구현 계획)을 확인해주세요. 미니멀하고 모바일 친화적인 디자인, 로컬 스토리지 기반의 간단한 습관 추적 앱에 대한 구조와 로직(함수 한글 주석 포함)을 계획했습니다. 확인 후 승인해주시면 바로 코딩을 시작하겠습니다!

implementation_plan.md Open Proceed

Copy

task.md ../../brain/bf4a4fbb-7b1f-4f27-82ae-c695e8238edf

Simple Habit Tracker Tasks

- [x] Create implementation plan and get user approval.
- [ ] Implement index.html (structure).
- [ ] Implement style.css (minimalist, mobile-friendly design).
- [ ] Implement script.js (logic, localStorage, Korean comments).
- [ ] Verify functionality in the browser.

Implement script.js (logic, localStorage, Korean comments). Comment on this line

Leave a comment

Cancel Add Comment

아주 간단한 습관 추적 앱을 만들어 주세요.
사용자가 습관을 추가하고, 오늘 완료로 표시할 수 있어야 하며, 모든 데이터는 localStorage를 사용하여 저장되어야 합니다.
디자인은 깔끔하고 미니멀하며, 모바일 친화적이어야 하고, 불필요한 추가 기능은 없어야 합니다.
Implementation Plan 문서를 작성해 주세요. 생성되는 모든 artifact 는 한국어로 작성해주세요.

implementation_plan.md ../../brain/bf4a4fbb-7b1f-4f27-82ae-c695e8238edf

간단한 습관 추적 앱 구현 계획

사용자가 습관을 추가하고, 오늘 완료 여부를 체크할 수 있는 미니멀하고 모바일 친
`localStorage`를 사용하여 저장됩니다.

요구사항 요약

- 습관 추가 기능
- 오늘 완료 표시 토글 기능
- `localStorage`를 이용한 데이터 저장 및 영속성 보장
- 미니멀하고 모바일 친화적인 UI 적용
- 불필요한 부가 기능 제외
- 룰: 모든 함수 앞에 한글 주석 작성

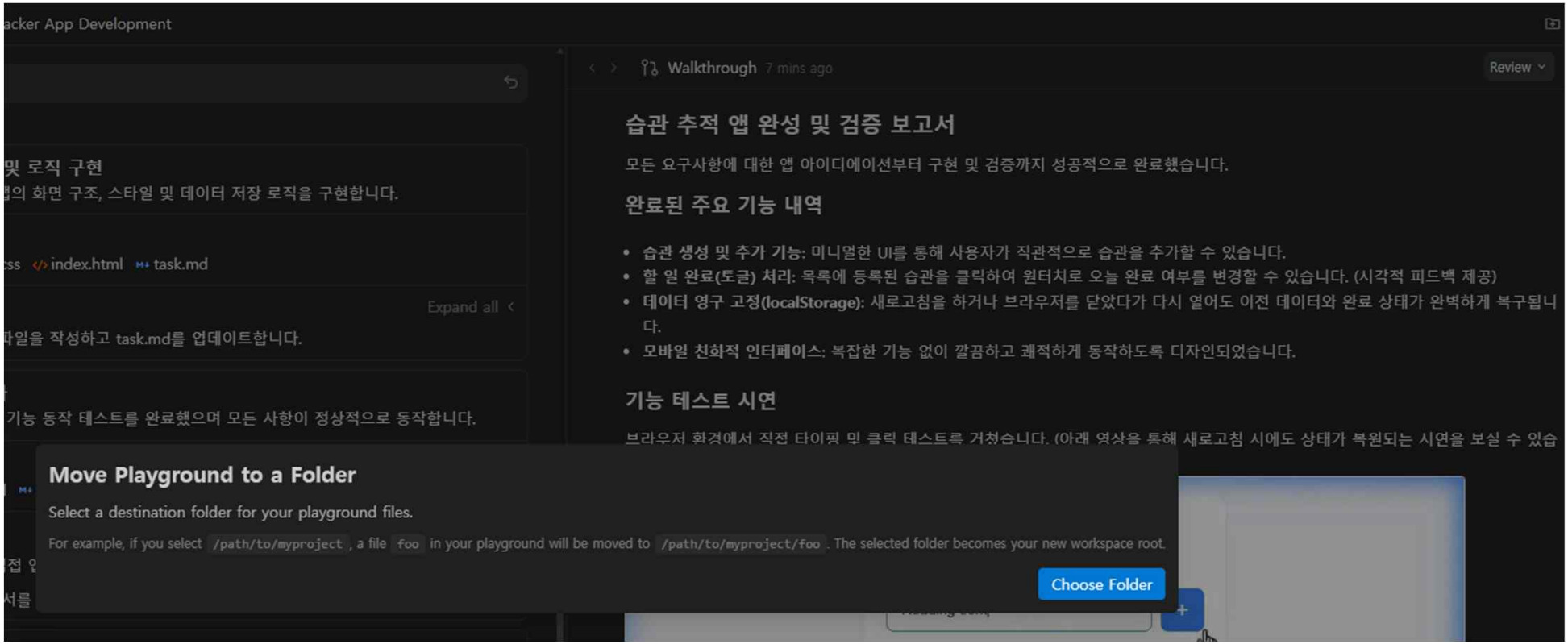
제안된 변경 사항

task.md ../../brain/bf4a4fbb-7b1f-4f27-82ae-c695e8238edf

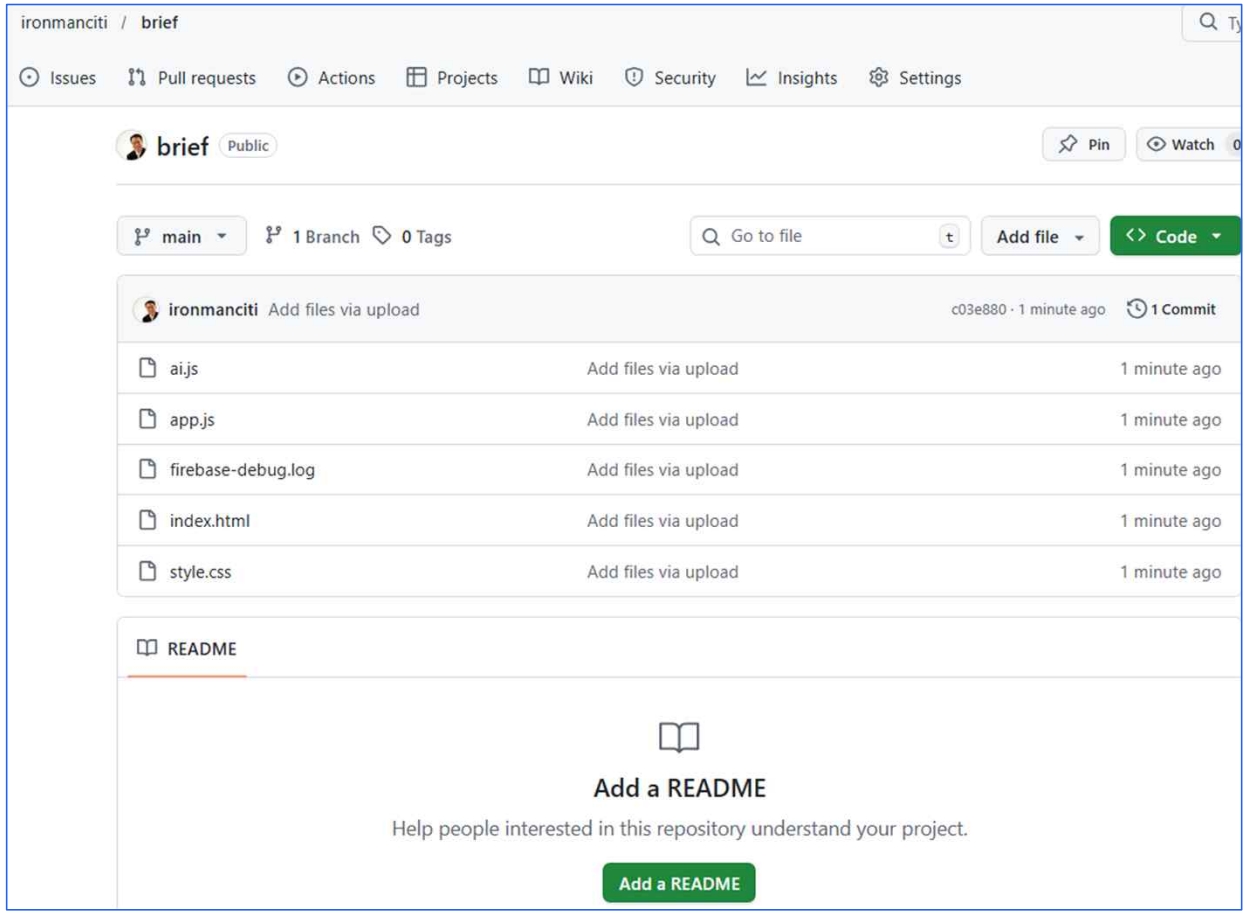
간단한 습관 추적 앱 개발

- [ ] 기획 및 설계
  - [x] 요구사항 분석 및 `implementation_plan.md` 작성
- [/] UI/UX 구현 (HTML/CSS)
  - [/] 깔끔하고 미니멀한 모바일 친화적 UI 레이아웃 작성 (`index.html`, `style.css`)
  - [/] 습관 추가 및 목록 표시 영역 구현
- [/] 로직 구현 (JavaScript)
  - [/] `localStorage`를 활용한 데이터 저장 및 불러오기 (`app.js`)
  - [/] 습관 추가 기능 구현
  - [/] 오늘 완료 표시 (토글) 기능 구현
- [ ] 테스트 및 안정화
  - [ ] 데이터 저장 검증 및 모바일 뷰 확인

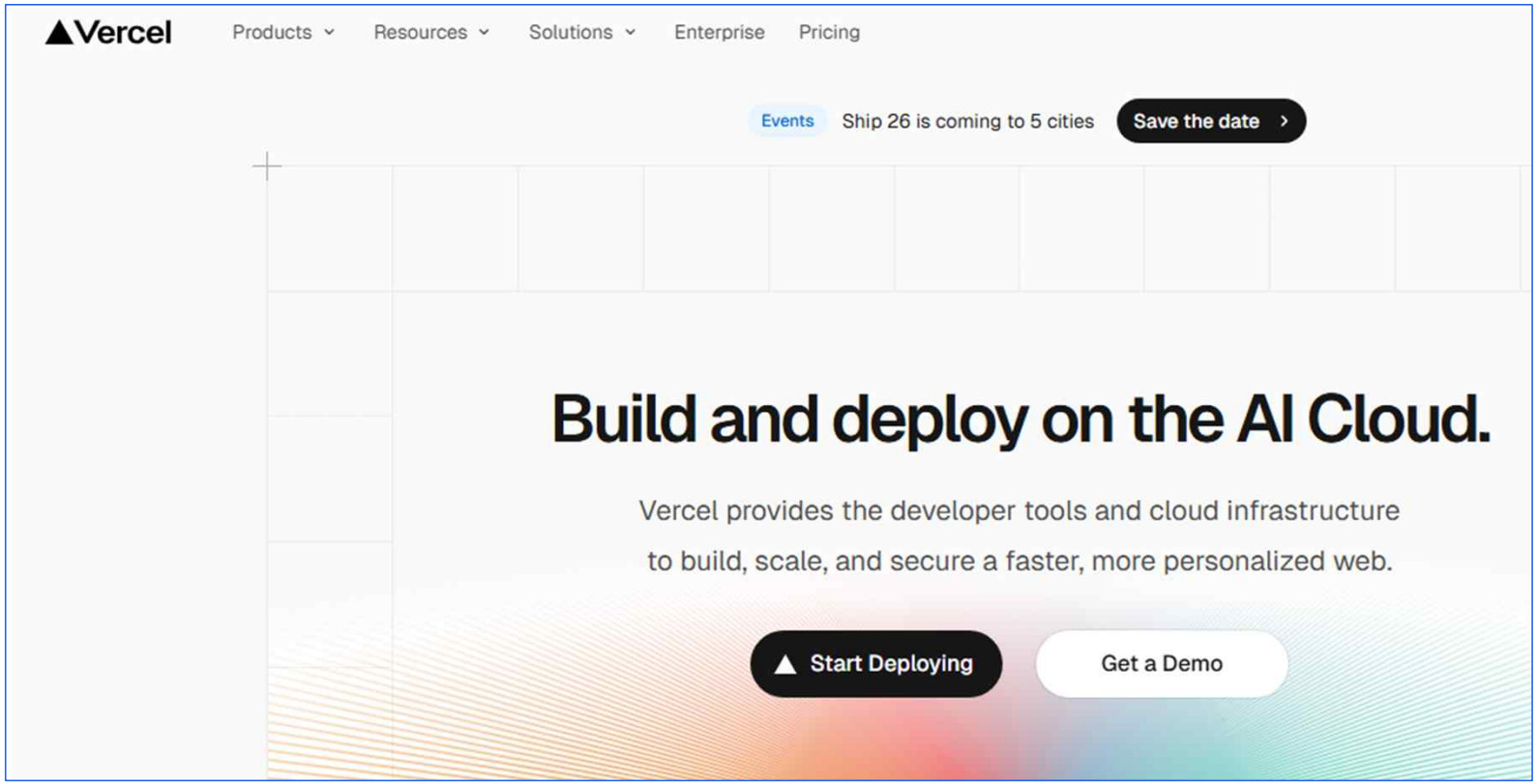
Playground → Folder

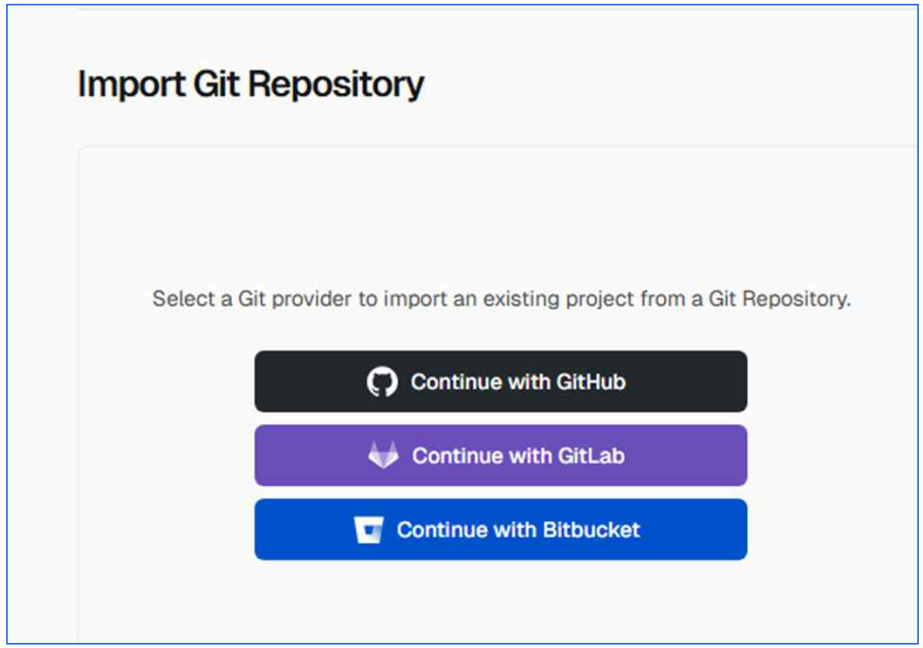


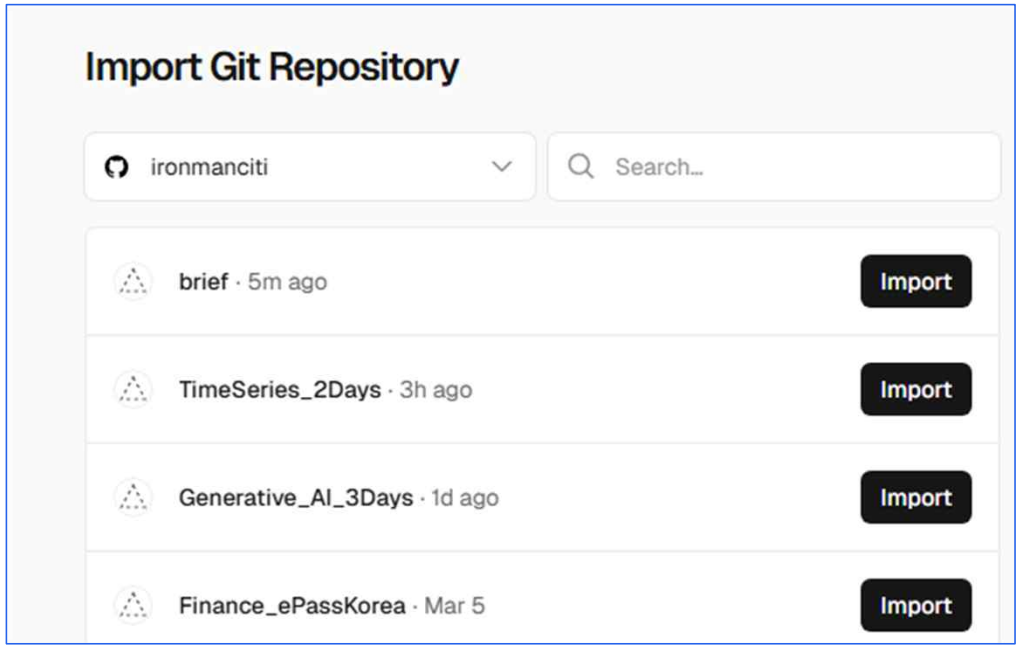
Deploy – git repository 생성

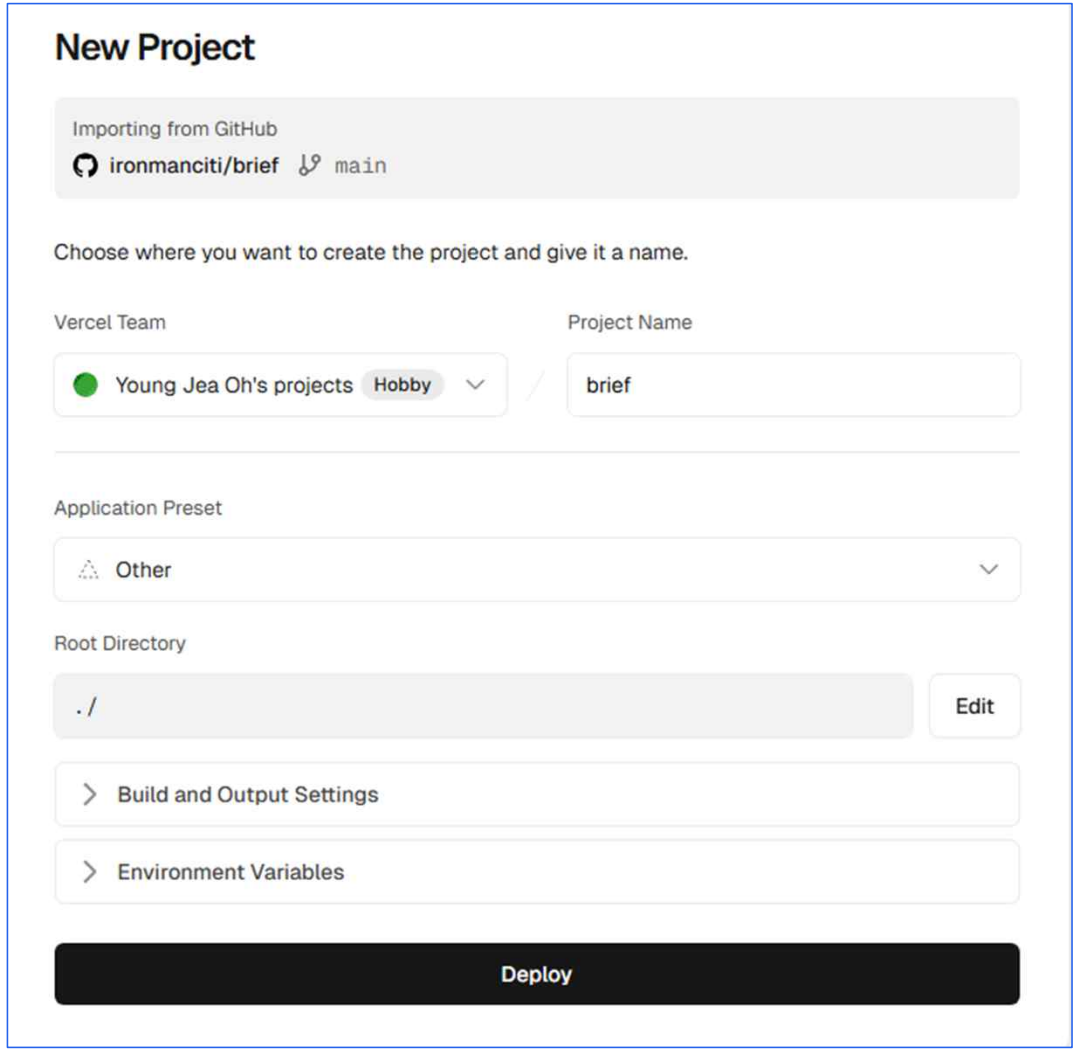


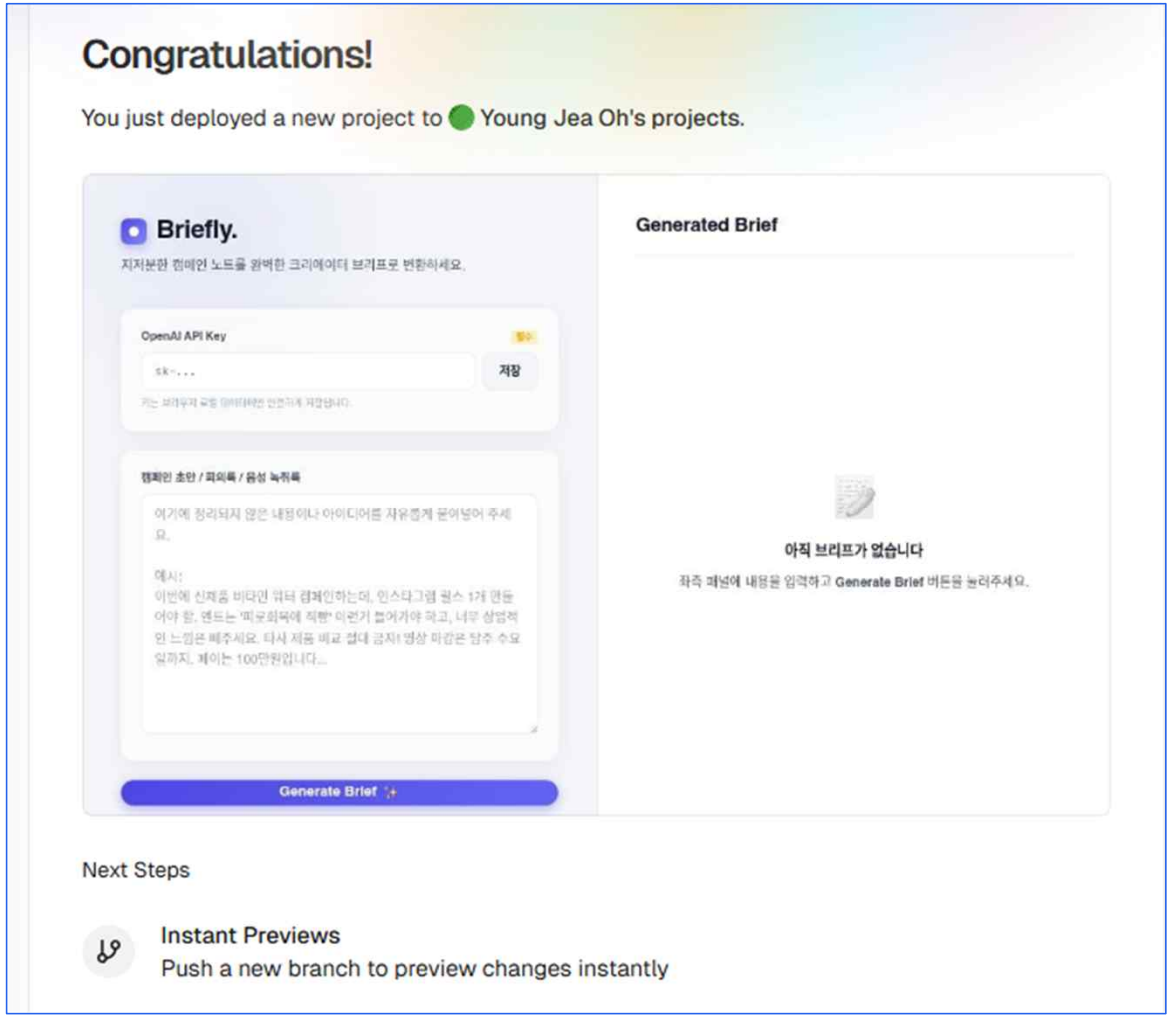
Vercel – easy deployment




Import Git Repository
Select a Git provider to import an existing project from a Git Repository.
Continue with GitHub
Continue with GitLab
Continue with Bitbucket


Import Git Repository
ironmanciti
Search...
brief · 5m ago
Import
TimeSeries_2Days · 3h ago
Import
Generative_AI_3Days · 1d ago
Import
Finance_ePassKorea · Mar 5
Import


New Project
Importing from GitHub
ironmanciti/brief
main
Choose where you want to create the project and give it a name.
Vercel Team
Project Name
Young Jea Oh's projects
Hobby
brief
Application Preset
Other
Root Directory
./
Edit
Build and Output Settings
Environment Variables
Deploy


Congratulations!
You just deployed a new project to Young Jea Oh's projects.
Briefly.
지저분한 캠페인 노트를 완벽한 크리에이터 브리프로 변환하세요.
OpenAI API Key
sk-...
저장
캠페인 초안 / 피의록 / 음성 녹취록
Generate Brief ✨
Generated Brief
아직 브리프가 없습니다
Next Steps
Instant Previews
Push a new branch to preview changes instantly